

第 9 章 动态规划初步

学习目标

- ☑ 理解状态和状态转移方程
- ☑ 理解最优子结构和重叠子问题
- ☑ 熟练运用递推法和记忆化搜索求解数字三角形问题
- ☑ 熟悉 DAG 上动态规划的常见思路、两种状态定义方法和刷表法
- ☑ 掌握记忆化搜索在实现方面的注意事项
- ☑ 掌握记忆化搜索和递推中输出方案的方法
- ☑ 掌握递推中滚动数组的使用方法
- ☑ 熟练解决经典动态规划问题

动态规划的理论性和实践性都比较强，一方面需要理解“状态”、“状态转移”、“最优子结构”、“重叠子问题”等概念，另一方面又需要根据题目的条件灵活设计算法。可以说，对动态规划的掌握情况在很大程度上能直接影响一个选手的分析和建模能力。

9.1 数字三角形

动态规划是一种用途很广的问题求解方法，它本身并不是一个特定的算法，而是一种思想，一种手段。下面通过一个题目阐述动态规划的基本思路和特点。

9.1.1 问题描述与状态定义

数字三角形问题。有一个由非负整数组成的三角形，第一行只有一个数，除了最下行之外每个数的左下方和右下方各有一个数，如图 9-1 所示。

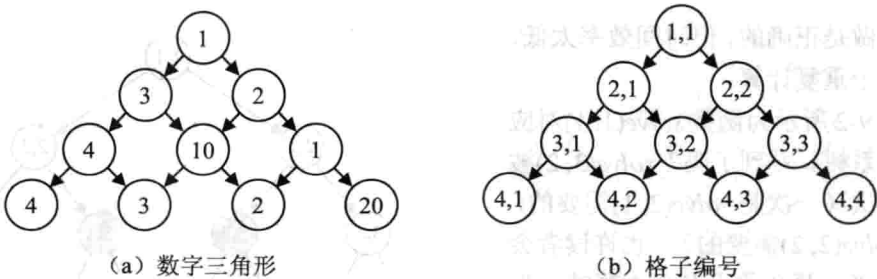


图 9-1 数字三角形问题

从第一行的数开始，每次可以往左下或右下走一格，直到走到最下行，把沿途经过的

数全部加起来。如何走才能使得这个和尽量大？

【分析】

如果熟悉回溯法，可能会立刻发现这是一个动态的决策问题：每次有两种选择——左下或右下。如果用回溯法求出所有可能的路线，就可以从中选出最优路线。但和往常一样，回溯法的效率太低：一个 n 层数字三角形的完整路线有 2^{n-1} 条，当 n 很大时回溯法的速度将让人无法忍受。

为了得到高效的算法，需要用抽象的方法思考问题：把当前的位置 (i, j) 看成一个状态（还记得吗？），然后定义状态 (i, j) 的指标函数 $d(i, j)$ 为从格子 (i, j) 出发时能得到的最大和（包括格子 (i, j) 本身的值）。在这个状态定义下，原问题的解是 $d(1, 1)$ 。

下面看看不同状态之间是如何转移的。从格子 (i, j) 出发有两种决策。如果往左走，则走到 $(i+1, j)$ 后需要求“从 $(i+1, j)$ 出发后能得到的最大和”这一问题，即 $d(i+1, j)$ 。类似地，往右走之后需要求解 $d(i+1, j+1)$ 。由于可以在这两个决策中自由选择，所以应选择 $d(i+1, j)$ 和 $d(i+1, j+1)$ 中较大的一个。换句话说，得到了所谓的状态转移方程：

$$d(i, j) = a(i, j) + \max\{d(i+1, j), d(i+1, j+1)\}$$

如果往左走，那么最好情况等于 (i, j) 格子里的值 $a(i, j)$ 与“从 $(i+1, j)$ 出发的最大总和”之和，此时需注意这里的“最大”二字。如果连“从 $(i+1, j)$ 出发走到底部”这部分的和都不是最大的，加上 $a(i, j)$ 之后肯定也不是最大的。这个性质称为最优子结构（optimal substructure），也可以描述成“全局最优解包含局部最优解”。不管怎样，状态和状态转移方程一起完整地描述了具体的算法。

提示 9-1：动态规划的核心是状态和状态转移方程。

9.1.2 记忆化搜索与递推

有了状态转移方程之后，应怎样计算呢？

方法 1：递归计算。程序如下（需注意边界处理）：

```
int solve(int i, int j){
    return a[i][j] + (i == n ? 0 : max(solve(i+1, j), solve(i+1, j+1)));
}
```

这样做是正确的，但时间效率太低，其原因在于重复计算。

如图 9-2 所示为函数 $\text{solve}(1, 1)$ 对应的调用关系树。看到了吗？ $\text{solve}(3, 2)$ 被计算了两次（一次是 $\text{solve}(2, 1)$ 需要的，一次是 $\text{solve}(2, 2)$ 需要的）。也许读者会认为重复算一两个数没有太大影响，但事实是：这样的重复不是单个结点，而是一棵子树。如果原来的三角形有 n 层，则调用关系树也会有 n 层，一共有 $2^n - 1$

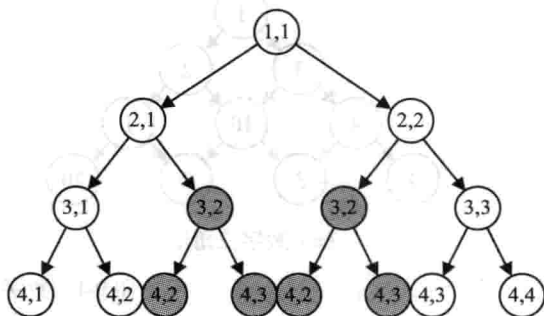


图 9-2 重叠子问题

个结点。

提示 9-2: 用直接递归的方法计算状态转移方程，效率往往十分低下。其原因是相同的子问题被重复计算了多次。

方法 2: 递推计算。程序如下（需再次注意边界处理）：

```
int i, j;
for(j = 1; j <= n; j++) d[n][j] = a[n][j];
for(i = n-1; i >= 1; i--)
    for(j = 1; j <= i; j++)
        d[i][j] = a[i][j] + max(d[i+1][j], d[i+1][j+1]);
```

程序的时间复杂度显然是 $O(n^2)$ ，但为什么可以这样计算呢？原因在于： i 是逆序枚举的，因此在计算 $d[i][j]$ 前，它所需要的 $d[i+1][j]$ 和 $d[i+1][j+1]$ 一定已经计算出来了。

提示 9-3: 可以用递推法计算状态转移方程。递推的关键是边界和计算顺序。在多数情况下，递推法的时间复杂度是：状态总数 $\times$ 每个状态的决策个数 $\times$ 决策时间。如果不同状态的决策个数不同，需具体问题具体分析。

方法 3: 记忆化搜索。程序分成两部分。首先用 “`memset(d, -1, sizeof(d));`” 把 d 全部初始化为 -1，然后编写递归函数^①：

```
int solve(int i, int j){
    if(d[i][j] >= 0) return d[i][j];
    return d[i][j] = a[i][j] + (i == n ? 0 : max(solve(i+1, j), solve(i+1, j+1)));
}
```

上述程序依然是递归的，但同时也把计算结果保存在数组 d 中。题目中说各个数都是非负的，因此如果已经计算过某个 $d[i][j]$ ，则它应是非负的。这样，只需把所有 d 初始化为 -1，即可通过判断是否 $d[i][j] \geq 0$ 得知它是否已经被计算过。

最后，千万不要忘记在计算之后把它保存在 $d[i][j]$ 中。根据 C 语言“赋值语句本身有返回值”的规定，可以把保存 $d[i][j]$ 的工作合并到函数的返回语句中。

上述程序的方法称为记忆化 (memoization)，它虽然不像递推法那样显式地指明了计算顺序，但仍然可以保证每个结点只访问一次，如图 9-3 所示。

由于 i 和 j 都在 $1 \sim n$ 之间，所有不相同的结点一共只有 $O(n^2)$ 个。无论以怎样的顺序访问，时

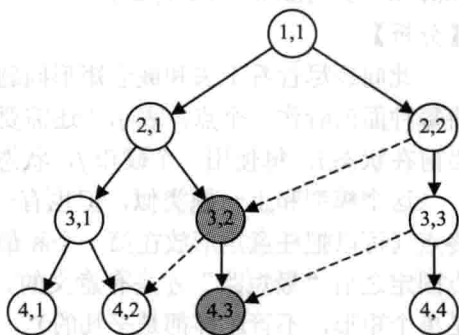


图 9-3 记忆化搜索

^① 注意这个函数的工作方式并不像它表面显示的那样——如果把 -1 改成 -2，并不是在把所有 d 值都初始化为 -2！请只用 0 和 -1 作为“批量赋值”的参数。

间复杂度均为 $O(n^2)$ 。从 $2^n \sim n^2$ 是一个巨大的优化，这正是利用了数字三角形具有大量重叠子问题的特点。

提示 9-4：可以用记忆化搜索的方法计算状态转移方程。当采用记忆化搜索时，不必事先确定各状态的计算顺序，但需要记录每个状态“是否已经计算过”。

9.2 DAG 上的动态规划

有向无环图上的动态规划是学习动态规划的基础。很多问题都可以转化为 DAG 上的最长路、最短路或路径计数问题。

9.2.1 DAG 模型

嵌套矩形问题。有 n 个矩形，每个矩形可以用两个整数 a, b 描述，表示它的长和宽。矩形 $X(a, b)$ 可以嵌套在矩形 $Y(c, d)$ 中，当且仅当 $a < c, b < d$ ，或者 $b < c, a < d$ （相当于把矩形 X 旋转 90° ）。例如， $(1, 5)$ 可以嵌套在 $(6, 2)$ 内，但不能嵌套在 $(3, 4)$ 内。你的任务是选出尽量多的矩形排成一行，使得除了最后一个之外，每一个矩形都可以嵌套在下一个矩形内。如果有多解，矩形编号的字典序应尽量小。

【分析】

矩形之间的“可嵌套”关系是一个典型的二元关系，二元关系可以用图来建模。如果矩形 X 可以嵌套在矩形 Y 里，就从 X 到 Y 连一条有向边。这个有向图是无环的，因为一个矩形无法直接或间接地嵌套在自己内部。换句话说，它是一个 DAG。这样，所要求的便是 DAG 上的最长路径。

硬币问题。有 n 种硬币，面值分别为 $V_1, V_2, \dots, V_n$ ，每种都有无限多。给定非负整数 S ，可以选用多少个硬币，使得面值之和恰好为 S ？输出硬币数目的最小值和最大值。 $1 \leq n \leq 100, 0 \leq S \leq 10000, 1 \leq V_i \leq S$ 。

【分析】

此问题尽管看上去和嵌套矩形问题很不一样，但本题的本质也是 DAG 上的路径问题。将每种面值看作一个点，表示“还需要凑足的面值”，则初始状态为 S ，目标状态为 0 。若当前在状态 i ，每使用一个硬币 j ，状态便转移到 $i - V_j$ 。

这个模型和上一题类似，但也有一些明显的不同之处：上题并没有确定路径的起点和终点（可以把任意矩形放在第一个和最后一个），而本题的起点必须为 S ，终点必须为 0 ；点固定之后“最短路”才是有意义的。在上题中，最短序列显然是空（如果不允许空，就是单个矩形，不管怎样都是平凡的），而本题的最短路却不容易确定。

9.2.2 最长路及其字典序

首先思考“嵌套矩形”。如何求 DAG 中不固定起点的最长路径呢？仿照数字三角形的

做法, 设 $d(i)$ 表示从结点 i 出发的最长路长度, 应该如何写状态转移方程呢? 第一步只能走到它的相邻点, 因此:

$$d(i) = \max\{d(j) + 1 \mid (i, j) \in E\}$$

其中, E 为边集。最终答案是所有 $d(i)$ 中的最大值。根据前面的介绍, 可以尝试按照递推或记忆化搜索的方式计算上式。不管怎样, 都需要先把图建立出来, 假设用邻接矩阵保存在矩阵 G 中 (在编写主程序之前需测试和调试程序, 以确保建图过程正确无误)。接下来编写记忆化搜索程序 (调用前需初始化 d 数组的所有值为 0):

```
int dp(int i) {
    int& ans = d[i];
    if(ans > 0) return ans;
    ans = 1;
    for(int j = 1; j <= n; j++)
        if(G[i][j]) ans = max(ans, dp(j)+1);
    return ans;
}
```

这里用到了一个技巧: 为表项 $d[i]$ 声明一个引用 ans 。这样, 任何对 ans 的读写实际上都是在对 $d[i]$ 进行。当 $d[i]$ 换成 $d[i][j][k][l][m][n]$ 这样很长的名字时, 该技巧的优势就会很明显。

提示 9-5: 在记忆化搜索中, 可以为正在处理的表项声明一个引用, 简化对它的读写操作。

原题还有一个要求: 如果有多个最优解, 矩形编号的字典序应最小。还记得第 6 章中的例题“理想路径”吗? 方法与其类似。将所有 d 值计算出来以后, 选择最大 $d[i]$ 所对应的 i 。如果有多个 i , 则选择最小的 i , 这样才能保证字典序最小。接下来可以选择 $d(i) = d(j) + 1$ 且 $(i, j) \in E$ 的任何一个 j 。为了让方案的字典序最小, 应选择其中最小的 j 。程序如下^①:

```
void print_ans(int i) {
    printf("%d ", i);
    for(int j = 1; j <= n; j++) if(G[i][j] && d[i] == d[j]+1){
        print_ans(j);
        break;
    }
}
```

提示 9-6: 根据各个状态的指标值可以依次确定各个最优决策, 从而构造出完整方案。由于决策是依次确定的, 所以很容易按照字典序打印出所有方案。

注意, 当找到一个满足 $d[i] = d[j] + 1$ 的结点 j 后就应立刻递归打印从 j 开始的路径, 并在递归返回后退出循环。如果要打印所有方案, 只把 `break` 语句删除是不够的 (想一想, 为什么)。正确的方法是记录路径上的所有点, 在递归结束时才一次性输出整条路径。程序

^① 输出的最后会有一个多余空格, 并且没有回车符。在使用时, 应在主程序调用 `print_ans` 后加一个回车符。如果比赛明确规定行末不允许有多余空格, 则可以像前面介绍的那样加一个变量 `first` 来帮助判断。

留给读者编写。

有趣的是,如果把状态定义成“ $d(i)$ 表示以结点 i 为终点的最长路径长度”,也能顺利求出最优值,却难以打印出字典序最小的方案。想一想,为什么?你能总结出一些规律吗?

9.2.3 固定终点的最长路和最短路

接下来考虑“硬币问题”。最长路和最短路的求法是类似的,下面只考虑最长路。由于终点固定, $d(i)$ 的确切含义变为“从结点 i 出发到结点 0 的最长路径长度”。下面是求最长路的代码:

```
int dp(int S) {
    int& ans = d[S];
    if(ans >= 0) return ans;
    ans = 0;
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

注意到区别了吗?由于在本题中,路径长度是可以为 0 的(S 本身可以是 0),所以不能再用 $d=0$ 表示“这个 d 值还没有算过”。相应地,初始化时也不能再把 d 全设为 0,而要设置为一个负值——在正常情况下是取不到的。常见的方法是用 -1 来表示“没有算过”,则初始化时只需用 `memset(d,-1, sizeof(d))` 即可。至此,已完整解释了上面的代码为什么把 `if(ans>0)` 改成了 `if(ans>=0)`。

提示 9-7: 当程序中需要用到特殊值时,应确保该值在正常情况下不会被取到。这不仅意味着特殊值不能有“正常的理解方式”,而且也不能在正常运算中“意外得到”。

不知读者有没有看出,上述代码有一个致命的错误,即由于结点 S 不一定真的能到达结点 0,所以需要用特殊的 $d[S]$ 值表示“无法到达”,但在上述代码中,如果 S 根本无法继续往前走,返回值是 0,将被误以为是“不用走,已经到达终点”的意思。如果把 `ans` 初始化为 -1 呢?别忘了 -1 代表“还没算过”,所以返回 -1 相当于放弃了自己的劳动成果。如果把 `ans` 初始化为一个很大的整数,例如 2^{30} 呢?如果一开始就这么大, `ans = max(ans, dp(i)+1)` 还能把 `ans` 变回“正常值”吗?如果改成很小的整数,例如 -2^{30} 呢?从目前来看,它也会被认为是“还没算过”,但至少可以和所有 d 的初值分开——只需把代码中 `if(ans>=0)` 改为 `if(ans!= -1)` 即可,如下所示:

```
int dp(int S){
    int& ans = d[S];
    if(ans != -1) return ans;
    ans = -(1<<30);
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

提示 9-8: 在记忆化搜索中, 如果用特殊值表示“还没算过”, 则必须将其和其他特殊值(如无解)区分开。

上述错误都是很常见的, 甚至“顶尖高手”有时也会一时糊涂, 掉入陷阱。意识到这些问题, 寻求解决方案是不难的, 但就怕调试很久以后仍然没有发现是哪出了问题。另一个解决方法是不用特殊值表示“还没算过”, 而用另外一个数组 `vis[i]` 表示状态 `i` 是否被访问过, 如下所示:

```
int dp(int S){
    if(vis[S]) return d[S];
    vis[S] = 1;
    int& ans = d[S];
    ans = -(1<<30);
    for(int i = 1; i <= n; i++) if(S >= V[i]) ans = max(ans, dp(S-V[i])+1);
    return ans;
}
```

尽管多了一个数组, 但可读性增强了许多: 再也不用担心特殊值之间的冲突了, 在任何情况下, 记忆化搜索的初始化都可以用 `memset(vis, 0, sizeof(vis))`^① 实现。

提示 9-9: 在记忆化搜索中, 可以用 `vis` 数组记录每个状态是否计算过, 以占用一些内存为代价增强程序的可读性, 同时减少出错的可能。

本题要求最小、最大两个值, 记忆化搜索就必须写两个。在这种情况下, 用递推更加方便(此时需注意递推的顺序):

```
minv[0] = maxv[0] = 0;
for(int i = 1; i <= S; i++){
    minv[i] = INF; maxv[i] = -INF;
}
for(int i = 1; i <= S; i++)
    for(int j = 1; j <= n; j++)
        if(i >= V[j]){
            minv[i] = min(minv[i], minv[i-V[j]] + 1);
            maxv[i] = max(maxv[i], maxv[i-V[j]] + 1);
        }
printf("%d %d\n", minv[S], maxv[S]);
```

如何输出字典序最小的方案呢? 刚刚介绍的方法仍然适用, 如下所示:

```
void print_ans(int* d, int S){
    for(int i = 1; i <= n; i++)
        if(S >= V[i] && d[S] == d[S-V[i]]+1){
```

^① 如果状态比较复杂, 推荐用 STL 中的 `map` 而不是普通数组保存状态值。这样, 判断状态 `S` 是否算过只需用 `if(d.count(S))` 即可。

```

printf("%d ", i);
print_ans(d, S-V[i]);
break;
}
}

```

然后分别调用 `print_ans(min, S)` (注意在后面要加一个回车符) 和 `print_ans(max, S)` 即可。输出路径部分和上题的区别是，上题打印的是路径上的点，而这里打印的是路径上的边。还记得数组可以作为指针传递吗？这里需要强调的一点是：数组作为指针传递时，不会复制数组中的数据，因此不必担心这样会带来不必要的时间开销。

提示 9-10：当用递推法计算出各个状态的指标之后，可以用与记忆化搜索完全相同的方式打印方案。

很多用户喜欢另外一种打印路径的方法：递推时直接用 `min_coin[S]` 记录满足 `min[S] == min[S-V[i]]+1` 的最小的 `i`，则打印路径时可以省去 `print_ans` 函数中的循环，并可以方便地把递归改成迭代（原来的也可以改成迭代，但不那么自然）。具体来说，需要把递推过程改成以下形式：

```

for(int i = 1; i <= S; i++)
    for(int j = 1; j <= n; j++)
        if(i >= V[j]){
            if(min[i] > min[i-V[j]] + 1){
                min[i] = min[i-V[j]] + 1;
                min_coin[i] = j;
            }
            if(max[i] < max[i-V[j]] + 1){
                max[i] = max[i-V[j]] + 1;
                max_coin[i] = j;
            }
        }
}

```

注意，判断中用的是“>”和“<”，而不是“>=”和“<=”，原因在于“字典序最小解”要求当 `min/max` 值相同时取最小的 `i` 值。反过来，如果 `j` 是从大到小枚举的，就需要把“>”和“<”改成“>=”和“<=”才能求出字典序最小解。

在求出 `min_coin` 和 `max_coin` 之后，只需调用 `print_ans(min_coin, S)` 和 `print_ans(max_coin, S)` 即可。

```

void print_ans(int* d, int S){
    while(S){
        printf("%d ", d[S]);
        S -= V[d[S]];
    }
}

```

该方法是一个“用空间换时间”的经典例子——用 `min_coin` 和 `max_coin` 数组消除了原来 `print_ans` 中的循环。

提示 9-11: 无论是用记忆化搜索还是递推，如果在计算最优值的同时“顺便”算出各个状态下的第一次最优决策，则往往能让打印方案的过程更加简单、高效。这是一个典型的“用空间换时间”的例子。

9.2.4 小结与应用举例

本节介绍了动态规划的经典应用：DAG 中的最长路和最短路。和 9.1 节中的数字三角形问题一样，DAG 的最长路和最短路都可以用记忆化搜索和递推两种实现方式。打印解时既可以根据 `d` 值重新计算出每一步的最优决策，也可以在动态规划时“顺便”记录下每一步的最优决策。

由于 DAG 最长（短）路的特殊性，有两种“对称”的状态定义方式。

状态 1：设 $d(i)$ 为从 i 出发的最长路，则 $d(i) = \max\{d(j) + 1 \mid (i, j) \in E\}$ 。

状态 2：设 $d(i)$ 为以 i 结束的最长路，则 $d(i) = \max\{d(j) + 1 \mid (j, i) \in E\}$ 。

如果使用状态 2，“硬币问题”就变得和“嵌套矩形问题”几乎一样了（唯一的区别是：“嵌套矩形问题”还需要取所有 $d(i)$ 的最大值）！9.2.3 节中有意介绍了比较麻烦的状态 1，主要是为了展示一些常见技巧和陷阱，实际比赛中不推荐使用。

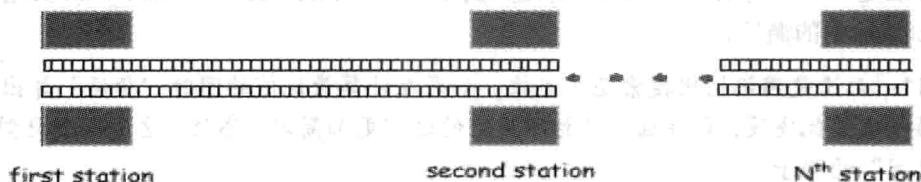
使用状态 2 时，有时还会遇到一个问题：状态转移方程可能不好计算，因为在很多时候，可以方便地枚举从某个结点 i 出发的所有边 (i, j) ，却不方便“反着”枚举 (j, i) 。特别是在有些题目中，这些边具有明显的实际背景，对应的过程不可逆。

这时需要用“刷表法”。什么是“刷表法”呢？传统的递推法可以表示成“对于每个状态 i ，计算 $f(i)$ ”，或者称为“填表法”。这需要对于每个状态 i ，找到 $f(i)$ 依赖的所有状态，在某些情况下并不方便。另一种方法是“对于每个状态 i ，更新 $f(i)$ 所影响到的状态”，或者称为“刷表法”。对应到 DAG 最长路的问题中，就相当于按照拓扑序枚举 i ，对于每个 i ，枚举边 (i, j) ，然后更新 $d[j] = \max(d[j], d[i] + 1)$ 。注意，一般不把这个式子叫做“状态转移方程”，因为它不是一个可以直接计算 $d[j]$ 的方程，而只是一个更新公式。

提示 9-12: 传统的递推法可以表示成“对于每个状态 i ，计算 $f(i)$ ”，或者称为“填表法”。这需要对于每个状态 i ，找到 $f(i)$ 依赖的所有状态，在某些时候并不方便。另一种方法是“对于每个状态 i ，更新 $f(i)$ 所影响到的状态”，或者称为“刷表法”，有时比填表法方便。但需要注意的是，只有当每个状态所依赖的状态对它的影响相互独立时才能用刷表法。

例题 9-1 城市里的间谍 (A Spy in the Metro, ACM/ICPC World Finals 2003, UVa1025)

某城市的地铁是线性的，有 n ($2 \leq n \leq 50$) 个车站，从左到右编号为 $1 \sim n$ 。有 $M1$ 辆列车从第 1 站开始往右开，还有 $M2$ 辆列车从第 n 站开始往左开。在时刻 0，Mario 从第 1 站出发，目的是在时刻 T ($0 \leq T \leq 200$) 会见车站 n 的一个间谍。在车站等车时容易被抓，所以她决定尽量躲在开动的火车上，让在车站等待的总时间尽量短。列车靠站停车时间忽略不计，且 Mario 身手敏捷，即使两辆方向不同的列车在同一时间靠站，Mario 也能完成换乘。



输入第 1 行为 n ，第 2 行为 T ，第 3 行有 $n-1$ 个整数 $t_1, t_2, \dots, t_{n-1}$ ($1 \leq t_i \leq 70$)，其中 t_i 表示地铁从车站 i 到 $i+1$ 的行驶时间（两个方向一样）。第 4 行为 $M1$ ($1 \leq M1 \leq 50$)，即从第 1 站出发向右开的列车数目。第 5 行包含 $M1$ 个整数 $d_1, d_2, \dots, d_{M1}$ ($0 \leq d_i \leq 250, d_i < d_{i+1}$)，即各列车的出发时间。第 6、7 行描述从第 n 站出发向左开的列车，格式同第 4、5 行。输出仅包含一行，即最少等待时间。无解输出 impossible。

【分析】

时间是单向流逝的，是一个天然的“序”。影响到决策的只有当前时间和所处的车站，所以可以用 $d(i, j)$ 表示时刻 i ，你在车站 j （编号为 $1 \sim n$ ），最少还需要等待多长时间。边界条件是 $d(T, n) = 0$ ，其他 $d(T, i)$ (i 不等于 n) 为正无穷。有如下 3 种决策。

决策 1：等 1 分钟。

决策 2：搭乘往右开的车（如果有）。

决策 3：搭乘往左开的车（如果有）。

主过程的代码如下：

```
for(int i = 1; i <= n-1; i++) dp[T][i] = INF;
dp[T][n] = 0;

for(int i = T-1; i >= 0; i--)
    for(int j = 1; j <= n; j++) {
        dp[i][j] = dp[i+1][j] + 1; //等待一个单位
        if(j < n && has_train[i][j][0] && i+t[j] <= T)
            dp[i][j] = min(dp[i][j], dp[i+t[j]][j+1]); //右
        if(j > 1 && has_train[i][j][1] && i+t[j-1] <= T)
            dp[i][j] = min(dp[i][j], dp[i+t[j-1]][j-1]); //左
    }

//输出
cout << "Case Number " << ++kase << ": ";
if(dp[0][1] >= INF) cout << "impossible\n";
else cout << dp[0][1] << "\n";
```

上面的代码中有一个 `has_train` 数组，其中 `has_train[t][i][0]` 表示时刻 t ，在车站 i 是否有往右开的火车，`has_train[t][i][1]` 类似，不过记录的是往左开的火车。这个数组不难在输入时计算处理，细节留给读者思考。

状态有 $O(nT)$ 个，每个状态最多只有 3 个决策，因此总时间复杂度为 $O(nT)$ 。

例题 9-2 巴比伦塔 (The Tower of Babylon, UVa 437)

有 n ($n \leq 30$) 种立方体, 每种都有无穷多个。要求选一些立方体摞成一根尽量高的柱子 (可以自行选择哪一条边作为高), 使得每个立方体的底面长宽分别严格小于它下方立方体的底面长宽。

【分析】

在任何时候, 只有顶面的尺寸会影响到后续决策, 因此可以用二元组 (a, b) 来表示“顶面尺寸为 $a \times b$ ”这个状态。因为每次增加一个立方体以后顶面的长和宽都会严格减小, 所以这个图是 DAG, 可以套用前面学过的 DAG 最长路算法。

这个算法没问题, 不过落实到程序上时会遇到一个问题: 不能直接用 $d(a, b)$ 表示状态值, 因为 a 和 b 可能会很大。怎么办呢? 可以用 (idx, k) 这个二元组来“间接”表达这个状态, 其中 idx 为顶面立方体的序号, k 是高的序号 (假设输入时把每个立方体的 3 个维度从小到大排序, 编号为 0~2)。例如, 若立方体 3 的大小为 $a \times b \times c$ (其中 $a \leq b \leq c$), 则状态 $(3, 1)$ 就是指这个立方体在顶面, 且高是 b (因此顶面大小为 $a \times c$)。因为 idx 是 $0 \sim n-1$ 的整数, k 是 $0 \sim 2$ 的整数, 所以可以很方便地用二维数组来存取。状态总数是 $O(n)$ 的, 每个状态的决策有 $O(n)$ 个, 时间复杂度为 $O(n^2)$ 。

例题 9-3 旅行 (Tour, ACM/ICPC SEERC 2005, UVa1347)

给定平面上 n ($n \leq 1000$) 个点的坐标 (按照 x 递增的顺序给出。各点 x 坐标不同, 且均为正整数), 你的任务是设计一条路线, 从最左边的点出发, 走到最右边的点后再返回, 要求除了最左点和最右点之外每个点恰好经过一次, 且路径总长度最短。两点间的长度为它们的欧几里德距离, 如图 9-4 所示。

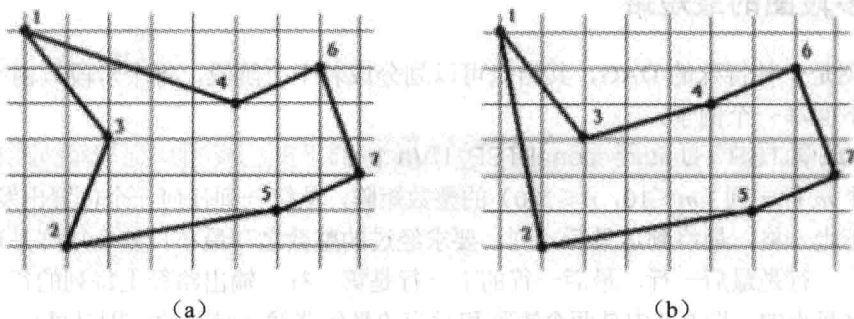


图 9-4 旅行路线示意图

【分析】

“从左到右再回来”不太方便思考, 可以改成: 两个人同时从最左点出发, 沿着两条不同的路径走, 最后都走到最右点, 且除了起点和终点外其余每个点恰好被一个人经过。这样, 就可以用 $d(i, j)$ 表示第一个人走到 i , 第二个人走到 j , 还需要走多长的距离。

状态如何转移呢? 仔细思考后会发现: 好像很难保证两个人不会走到相同的点。例如, 计算状态 $d(i, j)$ 时, 能不能让 i 走到 $i+1$ 呢? 不知道, 因为从状态里看不出来 $i+1$ 有没有被 j 走过。换句话说, 状态定义得不好, 导致转移困难。

下面修改一下: $d(i, j)$ 表示 $1 \sim \max(i, j)$ 全部走过, 且两个人的当前位置分别是 i 和 j , 还需

要走多长的距离。不难发现 $d(i,j)=d(j,i)$ ，因此从现在开始规定在状态中 $i>j$ 。这样，不管是哪个人，下一步只能走到 $i+1, i+2, \dots$ 这些点。可是，如果走到 $i+2$ ，情况变成了“1~ i 和 $i+2$ ，但是 $i+1$ 没走过”，无法表示成状态！怎么办？禁止这样的决策！也就是说，只允许其中一个人走到 $i+1$ ，而不能走到 $i+2, i+3, \dots$ 。换句话说，状态 $d(i,j)$ 只能转移到 $d(i+1,j)$ 和 $d(i+1,i)$ ^①。

可是这样做产生了一个问题：上述“霸道”的规定是否可能导致漏解呢？不会。因为如果第一个人直接走到了 $i+2$ ，那么它再也无法走到 $i+1$ 了，只能靠第二个人走到 $i+1$ 。既然如此，现在就让第二个人走到 $i+1$ ，并不会丢失解。

边界是 $d(n-1,j)=\text{dist}(n-1,n)+\text{dist}(j,n)$ ，其中 $\text{dist}(a,b)$ 表示点 a 和 b 之间的距离。因为根据定义，所有点都走过了，两个人只需直接走到终点。所求结果是 $\text{dist}(1,2)+d(2,1)$ ，因为第一步一定是某个人走到了第二个点，根据定义，这就是 $d(2,1)$ 。

状态总数有 $O(n^2)$ 个，每个状态的决策只有两个，因此总时间复杂度为 $O(n^2)$ 。

9.3 多阶段决策问题

还记得“多阶段决策问题”吗？在回溯法中曾提到过该问题。简单地说，每做一次决策就可以得到解的一部分，当所有决策做完之后，完整的解就“浮出水面”了。在回溯法中，每次决策对应于给一个结点产生新的子树，而解的生成过程对应一棵解答树，结点的层数就是“下一个待填充位置” cur 。

9.3.1 多段图的最短路

多段图是一种特殊的 DAG，其结点可以划分成若干个阶段，每个阶段只由上一个阶段所决定。下面举一个例子：

例题 9-4 单向 TSP (Unidirectional TSP, UVa 116)

给一个 m 行 n 列 ($m \leq 10, n \leq 100$) 的整数矩阵，从第一列任何一个位置出发每次往右、右上或右下走一格，最终到达最后一列。要求经过的整数之和最小。整个矩阵是环形的，即第一行的上一行是最后一行，最后一行的下一行是第一行。输出路径上每列的行号。多解时输出字典序最小的。图 9-5 中是两个矩阵和对应的最优路线（唯一的区别是最后一行）。

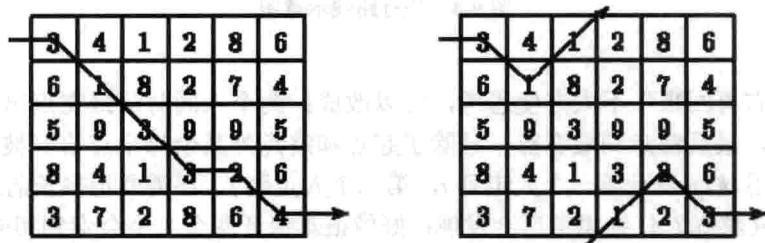


图 9-5 矩阵对应的最优路线

^① 第二个人走到 $i+1$ 时本应转移到 $d(i,i+1)$ ，但是根据此处规定，必须写成 $d(i+1,i)$ 。

【分析】

在这个题目中，每一列就是一个阶段，每个阶段都有3种决策：直行、右上和右下。

提示 9-13：多阶段决策的最优化问题往往可以用动态规划解决，其中，状态及其转移类似于回溯法中的解答树。解答树中的“层数”，也就是递归函数中的“当前填充位置”*cur*，描述的是即将完成的决策序号，在动态规划中被称为“阶段”。

有了前面的经验，不难设计出状态：设 $d(i, j)$ 为从格子 (i, j) 出发到最后一列的最小开销。但是本题不仅要输出解，还要求字典序最小，这就需要在计算 $d(i, j)$ 的同时记录“下一列的行号”的最小值（当然是在满足最优性的前提下），细节参见代码：

```
int ans = INF, first = 0;
for(int j = n-1; j >= 0; j--) { //逆推
    for(int i = 0; i < m; i++) {
        if(j == n-1) d[i][j] = a[i][j]; //边界
        else {
            int rows[3] = {i, i-1, i+1};
            if(i == 0) rows[1] = m-1; //第0行“上面”是第m-1行
            if(i == m-1) rows[2] = 0; //第m-1行“下面”是第0行
            sort(rows, rows+3); //重新排序，以便找到字典序最小的
            d[i][j] = INF;
            for(int k = 0; k < 3; k++) {
                int v = d[rows[k]][j+1] + a[i][j];
                if(v < d[i][j]) { d[i][j] = v; next[i][j] = rows[k]; }
            }
            if(j == 0 && d[i][j] < ans) { ans = d[i][j]; first = i; }
        }
    }
    printf("%d", first+1); //输出第1列
    for(int i = next[first][0], j = 1; j < n; i = next[i][j], j++)
        printf(" %d", i+1); //输出其他列
    printf("\n%d\n", ans);
}
return 0;
}
```

9.3.2 0-1 背包问题

0-1 背包问题是最广为人知的动态规划问题之一，拥有很多变形。尽管在理解之后并不难写出程序，但初学者往往需要较多的时间才能掌握它。在介绍 0-1 背包问题之前，先来看一个引例。

物品无限的背包问题。有 n 种物品，每种均有无穷多个。第 i 种物品的体积为 V_i ，重量

为 W_i 。选一些物品装到一个容量为 C 的背包中，使得背包内物品在总体积不超过 C 的前提下重量尽量大。 $1 \leq n \leq 100$, $1 \leq V_i \leq C \leq 10000$, $1 \leq W_i \leq 10^6$ 。

【分析】

很眼熟是吗？没错，它很像 9.2 节中的硬币问题，只不过“面值之和恰好为 S ”改成了“体积之和不超过 C ”，另外增加了一个新的属性——重量，相当于把原来的无权图改成了带权图（weighted graph）。这样，问题就变为了求以 C 为起点（终点任意）的、边权之和最大的路径。

与前面相比，DAG 从“无权”变成了“带权”，但这并没有带来任何困难，此时只需将某处代码从“+1”变成“+W[i]”即可。你能找到吗？

提示 9-14：动态规划的适用性很广。不少可以用动态规划解决的题目，在条件稍微变化后只需对状态转移方程做少量修改即可解决新问题。

0-1 背包问题。有 n 种物品，每种只有一个。第 i 种物品的体积为 V_i ，重量为 W_i 。选一些物品装到一个容量为 C 的背包，使得背包内物品在总体积不超过 C 的前提下重量尽量大。 $1 \leq n \leq 100$, $1 \leq V_i \leq C \leq 10000$, $1 \leq W_i \leq 10^6$ 。

【分析】

不知读者有没有发现，刚才的方法已经不适用了：只凭“剩余体积”这个状态，无法得知每个物品是否已经用过。换句话说，原来的状态转移太乱了，任何时候都允许使用任何一种物品，难以控制。为了消除这种混乱，需要让状态转移（也就是决策）有序化。

引入“阶段”之后，算法便不难设计了：用 $d(i, j)$ 表示当前在第 i 层，背包剩余容量为 j 时接下来的最大重量和，则 $d(i, j) = \max\{d(i+1, j), d(i+1, j - V[i]) + W[i]\}$ ，边界是 $i > n$ 时 $d(i, j) = 0$, $j < 0$ 时为负无穷（一般不会初始化这个边界，而是只当 $j \geq V[i]$ 时才计算第二项）。

说得更通俗一点， $d(i, j)$ 表示“把第 $i, i+1, i+2, \dots, n$ 个物品装到容量为 j 的背包中的最大总重量”。事实上，这个说法更加常用——“阶段”只是辅助思考的，在动态规划的状态描述中最好避免“阶段”、“层”这样的术语。很多教材和资料直接给出了这样的状态描述，而本书中则是花费了大量的篇幅叙述为什么会想到要划分阶段以及和回溯法的内在联系——如果对此理解不够深入，很容易出现“每次碰到新题自己都想不出来，但一看题解就懂”的尴尬情况。

提示 9-15：学习动态规划的题解，除了要理解状态表示及其转移方程外，最好思考一下为什么会想到这样的状态表示。

和往常一样，在得到状态转移方程之后，还需思考如何编写程序。尽管在很多情况下，记忆化搜索程序更直观、易懂，但在 0-1 背包问题中，递推法更加理想。为什么呢？因为有了“阶段”定义后，计算顺序变得非常明显。

提示 9-16：在多阶段决策问题中，阶段定义了天然的计算顺序。

下面是代码，答案是 $d[1][C]$ ：

```
for(int i = n; i >= 1; i--)
    for(int j = 0; j <= C; j++){
```

```

d[i][j] = (i==n ? 0 : d[i+1][j]);
if (j >= V[i]) d[i][j] = max(d[i][j], d[i+1][j-V[i]]+W[i]);
}

```

前面说过, i 必须逆序枚举, 但 j 的循环次序是无关紧要的。

规划方向。聪明的读者也许看出来了, 还有另外一种“对称”的状态定义: 用 $f(i, j)$ 表示“把前 i 个物品装到容量为 j 的背包中的最大总重量”, 其状态转移方程也不难得出:

$$f(i, j) = \max\{f(i-1, j), f(i-1, j-V[i]) + W[i]\}$$

边界是类似的: $i=0$ 时为 0, $j<0$ 时为负无穷, 最终答案为 $f(n, C)$ 。代码也是类似的:

```

for(int i = 1; i <= n; i++){
    for(int j = 0; j <= C; j++){
        f[i][j] = (i==1 ? 0 : f[i-1][j]);
        if(j >= V[i]) f[i][j] = max(f[i][j], f[i-1][j-V[i]]+W[i]);
    }
}

```

看上去这两种方式是完全对称的, 但其实存在细微区别: 新的状态定义 $f(i, j)$ 允许边读入边计算, 而不必把 V 和 W 保存下来。

```

for(int i = 1; i <= n; i++){
    scanf("%d%d", &V, &W);
    for(int j = 0; j <= C; j++){
        f[i][j] = (i==1 ? 0 : f[i-1][j]);
        if(j >= V) f[i][j] = max(f[i][j], f[i-1][j-V]+W);
    }
}

```

滚动数组。更奇妙的是, 还可以把数组 f 变成一维的:

```

memset(f, 0, sizeof(f));
for(int i = 1; i <= n; i++){
    scanf("%d%d", &V, &W);
    for(int j = C; j >= 0; j--){
        if(j >= V) f[j] = max(f[j], f[j-V]+W);
    }
}

```

为什么这样做是正确的呢? 下面来看一下 $f(i, j)$ 的计算过程, 如图 9-6 所示。

f 数组是从上到下、从右往左计算的。在计算 $f(i, j)$ 之前, $f[j]$ 里保存的就是 $f(i-1, j)$ 的值, 而 $f[j-W]$ 里保存的是 $f(i-1, j-W)$ 而不是 $f(i, j-W)$ ——别忘了 j 是逆序枚举的, 此时 $f(i, j-W)$ 还没有算出来。这样, $f[j] = (\max\{f[j], f[j-V]+W\})$ 实际上是把 $\max\{f(i-1, j), f(i-1, j-V)\}$ 保存在 $f[j]$ 中, 覆盖掉 $f[j]$ 原来的 $f(i-1, j)$ 。

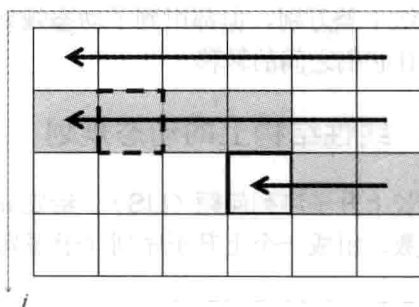


图 9-6 0-1 背包问题的计算顺序

提示 9-17: 在递推法中，如果计算顺序很特殊，而且计算新状态所用到的原状态不多，可以尝试用滚动数组减少内存开销。

滚动数组虽好，但也存在一些不尽如人意的地方，例如，打印方案较困难。当动态规划结束之后，只有最后一个阶段的状态值，而没有前面的值。不过这也不能完全归咎于滚动数组，规划方向也有一定责任——即使用二维数组，打印方案也不是特别方便。事实上，对于“前 i 个物品”这样的规划方向，只能用逆向的打印方案，而且还不能保证它的字典序最小（字典序比较是从前往后的）。

提示 9-18: 在使用滚动数组后，解的打印变得困难了，所以在需要打印方案甚至要求字典序最小方案的场合，应慎用滚动数组。

例题 9-5 劲歌金曲 (Jin Ge Jin Qu [h]ao, Rujia Liu's Present 6, UVa 12563)

如果问一个麦霸：“你在 KTV 里必唱的曲目有哪些？”得到的答案通常都会包含一首“神曲”：古巨基的《劲歌金曲》。为什么呢？一般来说，KTV 不会在“时间到”的时候鲁莽地把正在唱的歌切掉，而是会等它放完。例如，在还有 15 秒时再唱一首 2 分钟的歌，则实际上多唱了 105 秒。但是融合了 37 首歌曲的《劲歌金曲》长达 11 分 18 秒^①，如果唱这首，相当于多唱了 663 秒！

假定你正在唱 KTV，还剩 t 秒时间。你决定接下来只唱你最爱的 n 首歌（不含《劲歌金曲》）中的一些，在时间结束之前再唱一个《劲歌金曲》，使得唱的总曲目尽量多（包含《劲歌金曲》），在此前提下尽量晚的离开 KTV。

输入 n ($n \leq 50$)， t ($t \leq 10^9$) 和每首歌的长度（保证不超过 3 分钟^②），输出唱的总曲目以及时间总长度。输入保证所有 $n+1$ 首曲子的总长度严格大于 t 。

【分析】

虽说 $t \leq 10^9$ ，但由于所有 $n+1$ 首曲子的总长度严格大于 t ，实际上 t 不会超过 $180n+678$ 。这样就可以转化为 0-1 背包问题了。细节留给读者思考。

9.4 更多经典模型

本节介绍一些常见结构中的动态规划，序列、表达式、凸多边形和树。尽管它们的形式和解法千差万别，但都用到了动态规划的思想：从复杂的题目背景中抽象出状态表示，然后设计它们之间的转移。

9.4.1 线性结构上的动态规划

最长上升子序列问题 (LIS)。 给定 n 个整数 $A_1, A_2, \dots, A_n$ ，按从左到右的顺序选出尽量多的整数，组成一个上升子序列（子序列可以理解为：删除 0 个或多个数，其他数的顺序

^① 还有《劲歌金曲 2》和《劲歌金曲 3》，但本题不予考虑。

^② 显然大多数歌的长度都大于 3 分钟，但是 KTV 可以“切歌”，因此这里的“长度”实际上是指“想唱的时间长度”。

不变)。例如序列 1, 6, 2, 3, 7, 5, 可以选出上升子序列 1, 2, 3, 5, 也可以选出 1, 6, 7, 但前者更长。选出的上升子序列中相邻元素不能相等。

【分析】

设 $d(i)$ 为以 i 结尾的最长上升子序列的长度, 则 $d(i) = \max\{0, d(j) | j < i, A_j < A_i\} + 1$, 最终答案是 $\max\{d(i)\}$ 。如果 LIS 中的相邻元素可以相等, 把小于号改成小于等于号即可。上述算法的时间复杂度为 $O(n^2)$ 。《算法竞赛入门经典》中介绍了一种方法把它优化到 $O(n \log n)$, 有兴趣的读者可以自行阅读。

最长公共子序列问题 (LCS)。给两个子序列 A 和 B, 如图 9-7 所示。求长度最大的公共子序列。例如 1, 5, 2, 6, 8, 7 和 2, 3, 5, 6, 9, 8, 4 的最长公共子序列为 5, 6, 8 (另一个解是 2, 6, 8)。

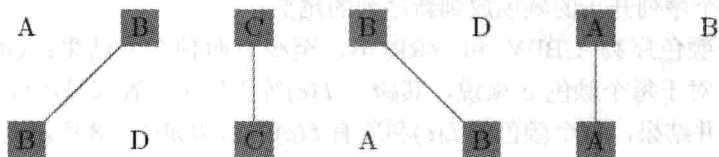


图 9-7 子序列 A 和 B

【分析】

设 $d(i, j)$ 为 $A_1, A_2, \dots, A_i$ 和 $B_1, B_2, \dots, B_j$ 的 LCS 长度, 则当 $A[i] = B[j]$ 时 $d(i, j) = d(i-1, j-1) + 1$, 否则 $d(i, j) = \max\{d(i-1, j), d(i, j-1)\}$, 时间复杂度为 $O(nm)$, 其中 n 和 m 分别是序列 A 和 B 的长度。

例题 9-6 照明系统设计 (Lighting System Design, UVa 11400)

你的任务是设计一个照明系统。一共有 n ($n \leq 1000$) 种灯泡可供选择, 不同种类的灯泡必须用不同的电源, 但同一种灯泡可以共用一个电源。每种灯泡用 4 个数值表示: 电压值 V ($V \leq 132000$), 电源费用 K ($K \leq 1000$), 每个灯泡的费用 C ($C \leq 10$) 和所需灯泡的数量 L ($1 \leq L \leq 100$)。

假定通过所有灯泡的电流都相同, 因此电压高的灯泡功率也更大。为了省钱, 可以把一些灯泡换成电压更高的另一种灯泡以节省电源的钱 (但不能换成电压更低的灯泡)。你的任务是计算出最优方案的费用。

【分析】

首先可以得到一个结论: 每种电压的灯泡要么全换, 要么全不换。因为如果只换部分灯泡, 如 $V=100$ 有两个灯泡, 把其中一个换成 $V=200$ 的, 另一个不变, 则 $V=100$ 和 $V=200$ 两种电源都需要, 不划算 (若一个都不换则只需要 $V=100$ 一种电源)。

先把灯泡按照电压从小到大排序。设 $s[i]$ 为前 i 种灯泡的总数量 (即 L 值之和), $d[i]$ 为灯泡 1~ i 的最小开销, 则 $d[i] = \min\{d[j] + (s[i] - s[j]) * c[i] + k[i]\}$, 表示前 j 个先用最优方案买, 然后第 $j+1$ ~ i 个都用第 i 号的电源。答案为 $d[n]$ 。

例题 9-7 划分成回文串 (Partitioning by Palindromes, UVa 11584)

输入一个由小写字母组成的字符串, 你的任务是把它划分成尽量少的回文串。例如, racecar 本身就是回文串; fastcar 只能分成 7 个单字母的回文串, aaadbccb 最少分成 3 个回

文串：aaa, d, bccb。字符串长度不超过 1000。

【分析】

$d[i]$ 为字符 $0 \sim i$ 划分成的最小回文串的个数，则 $d[i] = \min\{d[j] + 1 \mid s[j+1 \sim i] \text{ 是回文串}\}$ 。注意频繁的要判断回文串。状态 $O(n)$ 个，决策 $O(n)$ 个，如果每次转移都需要 $O(n)$ 时间判断，总时间复杂度会达到 $O(n^3)$ 。

可以先用 $O(n^2)$ 时间预处理 $s[i..j]$ 是否为回文串。方法是枚举中心，然后不断向左右延伸并且标记当前子串是回文串，直到延伸的左右字符不同为止。这样一来，每次转移的时间降为了 $O(1)$ ，总时间复杂度为 $O(n^2)$ 。

例题 9-8 颜色的长度 (Color Length, ACM/ICPC Daejeon 2011, UVa1625)

输入两个长度分别为 n 和 m ($n, m \leq 5000$) 的颜色序列，要求按顺序合并成同一个序列，即每次可以把一个序列开头的颜色放到新序列的尾部。

例如，两个颜色序列 GBBY 和 YRRGB，至少有两种合并结果：GBYBRYRGB 和 YRRGGBBYB。对于每个颜色 c 来说，其跨度 $L(c)$ 等于最大位置和最小位置之差。例如，对于上面两种合并结果，每个颜色的 $L(c)$ 和所有 $L(c)$ 的总和如图 9-8 所示。

Color	G	Y	B	R	Sum
$L(c)$: Scenario 1	7	3	7	2	19
$L(c)$: Scenario 2	1	7	3	1	12

图 9-8 每个颜色的 $L(c)$ 和 $L(c)$ 的总和

你的任务是找一种合并方式，使得所有 $L(c)$ 的总和最小。

【分析】

根据前面的经验，可以设 $d(i, j)$ 表示两个序列已经分别移走了 i 和 j 个元素，还需要多少费用。等一下！什么叫“还需要多少费用”呢？本题的指标函数（即需要最小化的函数）比较复杂。当某颜色第一次出现在最终序列中时，并不知道它什么时候会结束；而某个颜色的最后一个元素已经移到最终序列里时，又“忘记”了它是什么时候第一次出现的。

怎么办呢？如果记录每个颜色的第一次出现位置，状态会变得很复杂，时间也无法承受，所以只能把在指标函数的“计算方式”上想办法：不是等到一个颜色全部移完之后再算，而是每次累加。换句话说，当把一个颜色移到最终序列前，需要把所有“已经出现但还没结束”的颜色的 $L(c)$ 值加 1。更进一步地，因为并不关心每个颜色的 $L(c)$ ，所以只需要知道有多少种颜色已经开始但尚未结束。

例如，序列 GBBY 和 YRRGB，分别已经移走了 1 个和 3 个元素（例如，已经合并成了 YRRG）。下次再从序列 2 移走一个元素（即 G）时，Y 和 G 需要加 1。下次再从序列 1 移走一个元素（它是 B）时，只有 Y 需要加 1（因为 G 已经结束）。

这样，可以事先算出每个颜色在两个序列中的开始和结束位置，就可以在动态规划时在 $O(1)$ 时间内计算出状态 $d(i, j)$ 中“有多少个颜色已经出现但尚未结束”，从而在 $O(1)$ 时间内完成状态转移。状态总是为 $O(nm)$ 个，总时间复杂度也是 $O(nm)$ 。

最优矩阵链乘。 一个 $n \times m$ 矩阵由 n 行 m 列共 $n \times m$ 个数排列而成。两个矩阵 A 和 B 可以相乘当且仅当 A 的列数等于 B 的行数。一个 $n \times m$ 的矩阵乘以一个 $m \times p$ 的矩阵等于一

个 $n \times p$ 的矩阵, 运算量为 mnp 。

矩阵乘法不满足分配律, 但满足结合律, 因此 $A \times B \times C$ 既可以按顺序 $(A \times B) \times C$ 进行, 也可以按 $A \times (B \times C)$ 进行。假设 A 、 B 、 C 分别是 2×3 , 3×4 和 4×5 的, 则 $(A \times B) \times C$ 的运算量为 $2 \times 3 \times 4 + 2 \times 4 \times 5 = 64$, $A \times (B \times C)$ 的运算量为 $3 \times 4 \times 5 + 2 \times 3 \times 5 = 90$ 。显然第一种顺序节省运算量。

给出 n 个矩阵组成的序列, 设计一种方法把它们依次乘起来, 使得总的运算量尽量小。假设第 i 个矩阵 A_i 是 $p_{i-1} \times p_i$ 的。

【分析】

本题任务是设计一个表达式。在整个表达式中, 一定有一个“最后一次乘法”。假设它是第 k 个乘号, 则在此之前已经算出了 $P = A_1 \times A_2 \times \cdots \times A_k$ 和 $Q = A_{k+1} \times A_{k+2} \times \cdots \times A_n$ 。由于 P 和 Q 的计算过程互不相干, 而且无论按照怎样的顺序, P 和 Q 的值都不会发生改变, 因此只需分别让 P 和 Q 按照最优方案计算(最优子结构!)即可。为了计算 P 的最优方案, 还需要继续枚举 $P = A_1 \times A_2 \times \cdots \times A_k$ 的“最后一次乘法”, 把它分成两部分。不难发现, 无论怎么分, 在任意时候, 需要处理的子问题都形如“把 $A_i, A_{i+1}, \dots, A_j$ 乘起来需要多少次乘法?” 如果用状态 $f(i, j)$ 表示这个子问题的值, 不难列出如下的状态转移方程:

$$f(i, j) = \min \{f(i, k) + f(k+1, j) + p_{i-1}p_kp_j\}$$

边界为 $f(i, i) = 0$ 。上述方程有些特殊: 记忆化搜索固然没问题, 但如果要写成递推, 无论按照 i 还是 j 的递增或递减顺序均不正确。正确的方法是按照 $j-i$ 递增的顺序递推, 因为长区间的值依赖于短区间的值。

最优三角剖分。 对于一个 n 个顶点的凸多边形, 有很多种方法可以对它进行三角剖分(triangulation), 即用 $n-3$ 条互不相交的对角线把凸多边形分成 $n-2$ 个三角形。为每个三角形规定一个权函数 $w(i, j, k)$ (如三角形的周长或 3 个顶点的权和), 求让所有三角形权和最大的方案。

【分析】

本题和最优矩阵链乘问题十分相似, 但存在一个显著不同: 链乘表达式反映了决策过程, 而剖分不反映决策过程。举例来说, 在链乘问题中, 方案 $((A_1A_2)(A_3(A_4A_5)))$ 只能是先把序列分成 A_1A_2 和 $A_3A_4A_5$ 两部分, 而对于一个三角剖分, “第一刀”可以是任何一条对角线, 如图 9-9 所示。

如果允许随意切割, 则“半成品”多边形的各个顶点是可以在原多边形中随意选取的, 很难简洁定义成状态, 而“矩阵链乘”就不存在这个问题——无论怎样决策, 面临的子问题一定可以用区间表示。在这样的情况下, 有必要把决策的顺序规范化, 使得在规范的决策顺序下, 任意状态都能用区间表示。

定义 $d(i, j)$ 为子多边形 $i, i+1, \dots, j-1, j$ ($i < j$) 的最优值, 则边 $i-j$ 在最优解中一定对应一个三角形 $i-j-k$ ($i < k < j$), 如图 9-10 所示(注意顶点是按照逆时针编号的)。

因此, 状态转移方程为:

$$d(i, j) = \max \{d(i, k) + d(k, j) + w(i, j, k) \mid i < k < j\}$$

时间复杂度为 $O(n^3)$, 边界为 $d(i, i+1) = 0$, 原问题的解为 $d(0, n-1)$ 。

細心

例题 9-9 切木棍 (Cutting Sticks, UVa 10003)

【分析】

设 $d(i, j)$ 为切割小木棍 $i \sim j$ 的最优费用, 则 $d(i, j) = \min\{d(i, k) + d(k, j) \mid i < k < j\} + a[j] - a[i]$, 其中

状态有 $O(n^2)$ 个, 每个状态的决策有 $O(n)$ 个, 时间复杂度为 $O(n^3)$ 。值得一提的是, 本题可

例题 9-10 括号序列 (Brackets Sequence, NEERC 2001, UVa1626)

例题 9-10 括号序列 (Brackets Sequence, NEERC 2001, UVa1626)

定义如下正规括号序列（字符串）：

- 空序列是正规括号序列。
- 如果 S 是正规括号序列, 那么 (S) 和 $[S]$ 也是正规括号序列。
- 如果 A 和 B 都是正规括号序列, 那么 AB 也是正规括号序列。

输入一个长度不超过 100 的, 由“(”、“)”、“[”、“]”构成的序列, 添加尽量少

【分析】

设串 S 至少需要增加 $d(S)$ 个括号, 转移如下:

□ 如果 S 形如 (S') 或者 $[S']$, 转移到 $d(S')$ 。

- 如果 S 形如 (S') 或者 $[S']$, 转移到 $d(S')$ 。
- 如果 S 至少有两个字符, 则可以分成 AB , 转移到 $d(A)+d(B)$ 。

边界是: S 为时空 $d(S)=0$, S 为单字符时 $d(S)=1$ 。注意 $(S', [S',)S'$ 之类全部属于第二种转移, 不需要单独处理。

注意：不管 S 是否满足第一条，都要尝试第二种转移，否则“ $\square\square$ ”会转移到“ II ”，然后就只能加两个括号了。

当然，上述“方程”只是概念上的，落实到程序时要改成子串在原串中的起始点下标，即用 $d(i,j)$ 表示子串 $S[i \sim j]$ 至少需要添加几个括号。下面是递推写法，比记忆化写法要好几

倍, 而且代码更短。请读者注意状态的枚举顺序:

```
void dp() {
    for(int i = 0; i < n; i++) {
        d[i+1][i] = 0;
        d[i][i] = 1;
    }
    for(int i = n-2; i >= 0; i--)
        for(int j = i+1; j < n; j++) {
            d[i][j] = n;
            if(match(S[i], S[j])) d[i][j] = min(d[i][j], d[i+1][j-1]);
            for(int k = i; k < j; k++)
                d[i][j] = min(d[i][j], d[i][k] + d[k+1][j]);
        }
}
```

本题需要打印解, 但是上面的代码只计算了 d 数组, 如何打印解呢? 可以在打印时重新检查一下哪个决策最好。这样做的好处是节约空间, 坏处是打印时代码较复杂, 速度稍慢, 但是基本上可以忽略不计, 因为只有少数状态需要打印)。

```
void print(int i, int j) {
    if(i > j) return;
    if(i == j) {
        if(S[i] == '(' || S[i] == ')') printf("(");
        else printf("[");
        return;
    }
    int ans = d[i][j];
    if(match(S[i], S[j]) && ans == d[i+1][j-1]) {
        printf("%c", S[i]); print(i+1, j-1); printf("%c", S[j]);
        return;
    }
    for(int k = i; k < j; k++)
        if(ans == d[i][k] + d[k+1][j]) {
            print(i, k); print(k+1, j);
            return;
        }
}
```

本题唯一的陷阱是: 输入串可能是空串, 因此不能用 `scanf("%s", s)` 的方式输入, 只能用 `getchar`、`fgets` 或者 `getline`。

例题 9-11 最大面积最小的三角剖分 (Minimax Triangulation, ACM/ICPC NWERC 2004, UVa1331)

三角剖分是指用不相交的对角线把一个多边形分成若干个三角形。如图 9-11 所示是一

个六边形的几种不同的三角剖分。

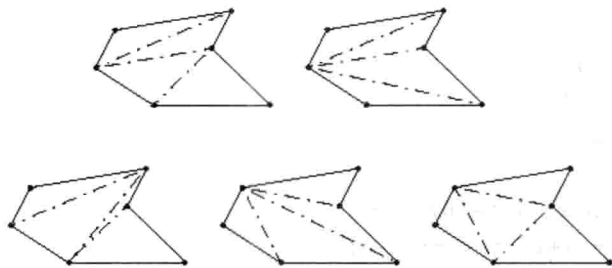


图 9-11 六边形的不同三角剖分

输入一个简单 m ($2 < m < 50$) 边形，找一个最大三角形面积最小的三角剖分。输出最大三角形的面积。在图 9-11 的 5 个方案中，最左边（即左下角）的方案最优。

【分析】

本题的程序实现要用到一些计算几何的知识，不过基本思想是清晰的：首先考虑凸多边形的简单情况。和“最优三角剖分”一样，设 $d(i, j)$ 为子多边形 $i, i+1, \dots, j-1, j$ ($i < j$) 的最优解，则状态转移方程为 $d(i, j) = \min\{S(i, j, k), d(i, k), d(k, j) \mid i < k < j\}$ ，其中 $S(i, j, k)$ 为三角形 $i-j-k$ 的面积。

回到原题。需要保证边 $i-j$ 是对角线^①（唯一的例外是 $i=0$ 且 $j=n-1$ ），具体方法是当边 $i-j$ 不满足条件时直接设 $d(i, j)$ 为无穷大，其他部分和凸多边形的情形完全一样。

9.4.2 树上的动态规划

树的最大独立集。对于一棵 n 个结点的无根树，选出尽量多的结点，使得任何两个结点均不相邻（称为最大独立集），然后输入 $n-1$ 条无向边，输出一个最大独立集（如果有多个解，则任意输出一组）。

【分析】

用 $d(i)$ 表示以 i 为根结点的子树的最大独立集大小。此时需要注意的是，本题的树是无根的：没有所谓的“父子”关系，而只有一些无向边。没关系，只要任选一个根 r ，无根树就变成了有根树，上述状态定义也就有意义了。

结点 i 只有两种决策：选和不选。如果不选 i ，则问题转化为了求出 i 的所有儿子的 d 值再相加；如果选 i ，则它的儿子全部不能选，问题转化为了求出 i 的所有孙子的 d 值之和。换句话说，状态转移方程为：

$$d(i) = \max\{1 + \sum_{j \in gs(i)} d(j), \sum_{j \in s(i)} d(j)\}$$

其中， $gs(i)$ 和 $s(i)$ 分别为 i 的孙子集合与儿子集合，如图 9-12 所示。

代码应如何编写呢？上面的方程涉及“枚举结点 i 的所有儿子和所有孙子”，颇为不便。其实可以换一个角度来看：不从 i 找 $s(i)$ 和 $gs(i)$ 的元素，而从 $s(i)$ 和 $gs(i)$ 的元素找 i 。换句话说，当计算出一个 $d(i)$ 后，用它去更新 i 的父亲和祖父结点的累加值 $\sum_{j \in gs(i)} d(j)$ 和 $\sum_{j \in s(i)} d(j)$ 。

^① 如何判断 $i-j$ 是否为多边形的对角线？限于篇幅，本书没有对计算几何进行专门讨论，请读者参考《算法竞赛入门经典——训练指南》的几何部分。

这样一来, 每个结点甚至不必记录其子结点有哪些, 只需记录父结点即可。这就是前面提过的“刷表法”。不过这个问题还有另外一种解法, 在实践中更加常用, 将在例题部分介绍。

树的重心(质心)。对于一棵 n 个结点的无根树, 找到一个点, 使得把树变成以该点为根的有根树时, 最大子树的结点数最小。换句话说, 删除这个点后最大连通块(一定是树)的结点数最小。

【分析】

和树的最大独立集问题类似, 先任选一个结点作为根, 把无根树变成有根树, 然后设 $d(i)$ 表示以 i 为根的子树的结点数。不难发现 $d(i) = \sum_{j \in s(i)} d(j) + 1$ 。程序实现也很简单: 只需要一次 DFS, 在无根树转有根树的同时计算即可, 连记忆化都不需要——因为本来就没有重复计算。

那么, 删除结点 i 后, 最大的连通块有多少个结点呢? 结点 i 的子树中最大的有 $\max\{d(j)\}$ 个结点, i 的“上方子树”中有 $n-d(i)$ 个结点, 如图 9-13 所示。这样, 在动态规划的过程中就可以顺便找出树的重心了。

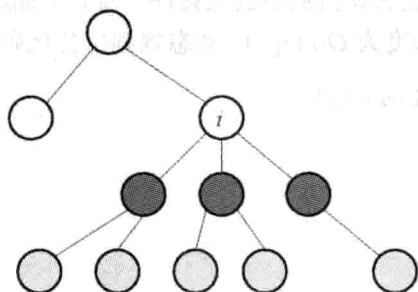


图 9-12 结点 i 的 $gs(i)$ (浅灰色) 和 $s(i)$ (深灰色)

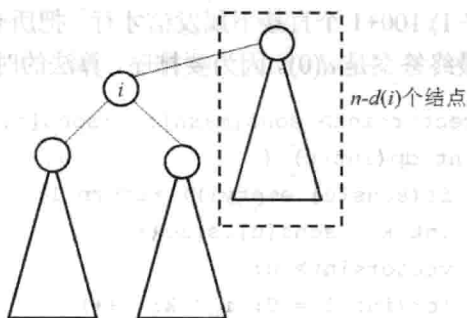


图 9-13 树中的结点分布

树的最长路径(最远点对)。对于一棵 n 个结点的无根树, 找到一条最长路径。换句话说, 要找到两个点, 使得它们的距离最远。

【分析】

和树的重心问题一样, 先把无根树转成有根树。对于任意结点 i , 经过 i 的最长路就是连接 i 的两棵不同子树 u 和 v 的最深叶子的路径, 如图 9-14 所示。

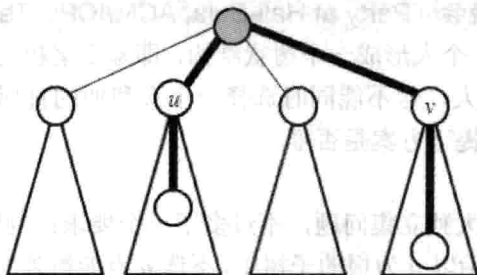


图 9-14 子树 u 和 v 的最深叶子路径

设 $d(i)$ 表示根为结点 i 的子树中根到叶子的最大距离, 不难写出状态转移方程:

$d(i) = \max\{d(j)\} + 1$ 。对于每个结点 i ，把所有子结点的 $d(j)$ 都求出来之后，设 d 值前两大

的结点为 u 和 v ，则 $d(u) + d(v) + 2$ 就是所求。

本题还有一个不用动态规划的解法：随便找一个结点 u ，用 DFS 求出 u 的最远结点 v ，然后再用一次 DFS 求出 v 的最远结点 w ，则 $v-w$ 就是最长路径。

结合上述两个问题的解法，可以解决下面的问题：对于一棵 n 个结点的无根树，求出每个结点的最远点，要求时间复杂度为 $O(n)$ 。这个问题留给读者思考。

例题 9-12 工人的请愿书 (Another Crisis, UVa 12186)

某公司里有一个老板和 n ($n \leq 10^5$) 个员工组成树状结构，除了老板之外每个员工都有唯一的直属上司。老板的编号为 0，员工编号为 $1 \sim n$ 。工人们（即没有直接下属的员工）打算签署一项请愿书递给老板，但是不能跨级递，只能递给直属上司。当一个中级员工（不是工人的员工）的直属下属中不小于 $T\%$ 的人签字时，他也会签字并且递给他的直属上司。问：要让公司老板收到请愿书，至少需要多少个工人签字？

【分析】

设 $d(u)$ 表示让 u 给上级发信最少需要多少个工人。假设 u 有 k 个子结点，则至少需要 $c = (kT - 1) / 100 + 1$ 个直接下属发信才行。把所有子结点的 d 值从小到大排序，前 c 个加起来即可。最终答案是 $d(0)$ 。因为要排序，算法的时间复杂度为 $O(n \log n)$ 。动态规划部分代码如下：

```
vector<int> sons[maxn]; //sons[i] 为结点 i 的子列表
int dp(int u) {
    if(sons[u].empty()) return 1;
    int k = sons[u].size();
    vector<int> d;
    for(int i = 0; i < k; i++)
        d.push_back(dp(sons[u][i]));
    sort(d.begin(), d.end());
    int c = (k*T - 1) / 100 + 1;
    int ans = 0;
    for(int i = 0; i < c; i++) ans += d[i];
    return ans;
}
```

例题 9-13 Hali-Bula 的晚会 (Party at Hali-Bula, ACM/ICPC Tehran 2006, UVa1220)

公司里有 n ($n \leq 200$) 个人形成一个树状结构，即除了老板之外每个员工都有唯一的直属上司。要求选尽量多的人，但不能同时选择一个人和他的直属上司。问：最多能选多少人，以及在人数最多的前提下方案是否唯一。

【分析】

本题几乎就是树的最大独立集问题，不过多了一个要求：判断唯一性。设：

- $d(u, 0)$ 和 $f(u, 0)$ 表示以 u 为根的子树中，不选 u 点能得到的最大人数以及方案唯一性 ($f(u, 0) = 1$ 表示唯一，0 表示不唯一)。
- $d(u, 1)$ 和 $f(u, 1)$ 表示以 u 为根的子树中，选 u 点能得到的最大人数以及方案唯一性。

相应地, 状态转移方程也有两套。

- $d(u,1)$ 的计算: 因为选了 u , 所以 u 的子结点都不能选, 因此 $d(u,1) = \sum \{d(v,0) \mid v \text{ 是 } u \text{ 的子结点}\}$ 。当且仅当所有 $f(v,0)=1$ 时 $f(u,1)$ 才是 1。
- $d(u,0)$ 的计算: 因为 u 没有选, 所以每个子结点 v 可选可不选, 即 $d(u,0) = \sum \{ \max(d(v,0), d(v,1)) \}$ 。什么情况下方案是唯一的呢? 首先, 如果某个 $d(v,0)$ 和 $d(v,1)$ 相等, 则不唯一; 其次, 如果 $\max$ 取到的那个值对应的 $f=0$, 方案也不唯一 (如 $d(v,0) > d(v,1)$ 且 $f(v,0)=0$, 则 $f(u,0)=0$)。

例题 9-14 完美的服务 (Perfect Service, ACM/ICPC Kaoshiung 2006, UVa1218)

有 n ($n \leq 10000$) 台机器形成树状结构。要求在其中一些机器上安装服务器, 使得每台不是服务器的计算机恰好和一台服务器计算机相邻。求服务器的最少数量。如图 9-15 所示, 图 9-15 (a) 是非法的, 因为 4 同时和两台服务器相邻, 而 6 不与任何一台服务器相邻。而图 9-15 (b) 是合法的。

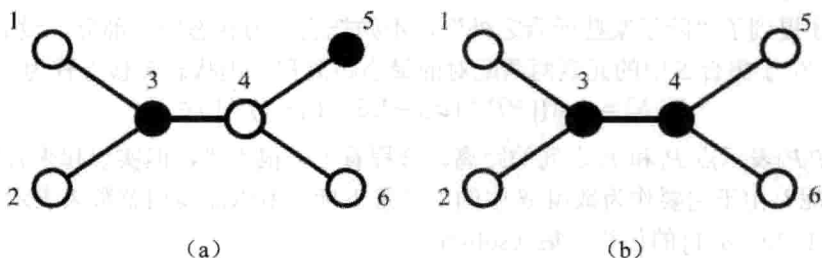


图 9-15 非法与合法的树状结构

【分析】

有了前面的经验, 这次仍然按照每个结点的情况进行分类。

- $d(u,0)$: u 是服务器, 则每个子结点可以是服务器也可以不是。
- $d(u,1)$: u 不是服务器, 但 u 的父亲是服务器, 这意味着 u 的所有子结点都不是服务器。
- $d(u,2)$: u 和 u 的父亲都不是服务器。这意味着 u 恰好有一个儿子是服务器。

状态转移比前面复杂一些, 但也不困难。首先可以写出:

$$d(u,0) = \sum \{ \min(d(v,0), d(v,1)) \} + 1$$

$$d(u,1) = \sum (d(v,2))$$

而 $d(u,2)$ 稍微复杂一点, 需要枚举当服务器的子结点编号 v , 然后把其他所有子结点 v' 的 $d(v',2)$ 加起来, 再和 $d(v,0)$ 相加。不过如果这样做, 每次枚举 v 都需要 $O(k)$ 时间 (其中 k 是 u 的子结点数目), 而 v 本身要枚举 k 次, 因此计算 $d(u,2)$ 需要花 $O(k^2)$ 时间。

刚才的做法有很多重复计算, 其实可以利用已经算出的 $d(u,1)$ 写出一个新的状态转移方程:

$$d(u,2) = \min(d(u,1) - d(v,2) + d(v,0))$$

这样一来, 计算 $d(u,2)$ 的时间复杂度变为了 $O(k)$ 。因为每个结点只有在计算父亲时被用了 3 次, 总时间复杂度为 $O(n)$ 。

9.4.3 复杂状态的动态规划

最优配对问题。空间里有 n 个点 $P_0, P_1, \dots, P_{n-1}$, 你的任务是把它们配成 $n/2$ 对 (n 是偶数), 使得每个点恰好在一个点对中。所有点对中两点的距离之和应尽量小。 $n \leq 20$, $|x_i|, |y_i|, |z_i| \leq 10000$ 。

【分析】

既然每个点都要配对, 很容易把问题看成如下的多阶段决策过程: 先确定 P_0 和谁配对, 然后是 P_1 , 接下来是 P_2 , …… , 最后是 P_{n-1} 。按照前面的思路, 设 $d(i)$ 表示把前 i 个点两两配对的最小距离和, 然后考虑第 i 个点的决策——它和谁配对呢? 假设它和点 j 配对 ($j < i$), 那么接下来的问题应是“把前 $i-1$ 个点中除了 j 之外的其他点两两配对”, 它显然无法用任何一个 d 值来刻画——此处的状态定义无法体现出“除了一些点之外”这样的限制。

当发现状态无法转移后, 常见的方法是增加维度, 即增加新的因素, 更细致地描述状态。既然刚才提到了“除了某些元素之外”, 不妨把它作为状态的一部分, 设 $d(i, S)$ 表示把前 i 个点中, 位于集合 S 中的元素两两配对的最小距离和, 则状态转移方程为:

$$d(i, S) = \min \{ |P_i P_j| + d(i-1, S - \{i\} - \{j\}) \mid j \in S \}$$

其中, $|P_i P_j|$ 表示点 P_i 和 P_j 之间的距离。方程看上去很不错, 但实现起来有问题: 如何表示集合 S 呢? 由于它要作为数组 d 中的第二维下标, 所以需要整数来表示集合, 确切地说, 是 $\{0, 1, 2, \dots, n-1\}$ 的任意子集 (subset)。

在第7章的“子集枚举”部分, 曾介绍过子集的二进制表示, 现在再次用到此知识:

```
for(int i = 0; i < n; i++)
    for(int S = 0; S < (1<<n); S++) {
        d[i][S] = INF;
        for(int j = 0; j < i; j++) if(S & (1<<j))
            d[i][S] = min(d[i][S], dist(i, j) + d[i-1][S^(1<<i)^(1<<j)]);
    }
```

上述程序中故意用了很多括号, 传达给读者的信息是: 位运算的优先级低, 初学者很容易弄错。例如, “ $1<<n-1$ ”的正确解释是“ $1<<(n-1)$ ”, 因为减法的优先级比左移要高。为了保险起见, 应多用括号。另一个技巧是利用C语言中“0为假, 非0为真”的规定简化表达式: “ $\text{if}(S \& (1<<j))$ ”的实际含义是“ $\text{if}((S \& (1<<j)) \neq 0)$ ”。

提示 9-19: 位运算的优先级往往比较低。如果不确定表达式的计算顺序, 应多用括号。

由于大量使用了形如 $1<<n$ 的表达式, 此类表达式中, 左移运算符“ $<<$ ”的含义是“把各个位往左移动, 右边补0”。根据二进制运算法则, 每次左移一位就相当于乘以2, 因此 $a<<b$ 相当于 $a*2^b$, 而在集合表示法中, $1<<i$ 代表单元素集合 $\{i\}$ 。由于0表示空集, “ $S \& (1<<j)$ ”不等于0就意味着“ S 和 $\{j\}$ 的交集不为空”。

上面的方程可以进一步简化。事实上, 阶段 i 根本不用保存, 它已经隐含在 S 中了—— S 中的最大元素就是 i 。这样, 可直接用 $d(S)$ 表示“把 S 中的元素两两配对的最小距离和”,

则状态转移方程为:

$$d(S) = \min\{|P_i P_j| + d(S - \{i\} - \{j\}) \mid j \in S, i = \max\{S\}\}$$

状态有 2^n 个, 每个状态有 $O(n)$ 种转移方式, 总时间复杂度为 $O(n2^n)$ 。

提示 9-20: 如果用二进制表示子集并进行动态规划, 集合中的元素就隐含了阶段信息。例如, 可以把集合中的最大元素想象成“阶段”。

值得一提的是, 不少用户一直在用这样的状态转移方程:

$$d(S) = \min\{|P_i P_j| + d(S - \{i\} - \{j\}) \mid i, j \in S\}$$

它和刚才的方程很类似, 唯一的不同是: i 和 j 都是需要枚举的。这样做虽然也没错, 但每个状态的转移次数高达 $O(n^2)$, 总时间复杂度为 $O(n^2 2^n)$, 比刚才的方法慢。这个例子再次说明: 即使用相同的状态描述, 减少决策也是很重要的。

提示 9-21: 即使状态定义相同, 过多地考虑不必要的决策仍可能会导致时间复杂度上升。

接下来出现了一个新问题: 如何求出 S 中的最大元素呢? 用一个循环判断即可。当 S 取遍 $\{0, 1, 2, \dots, n-1\}$ 的所有子集时, 平均判断次数仅为 2 (想一想, 为什么)。

```
for(int S = 0; S < (1<<n); S++) {
    int i, j;
    d[S] = INF;
    for(i = 0; i < n; i++)
        if(S & (1<<i)) break;
    for(j = i+1; j < n; j++)
        if(S & (1<<j)) d[S] = max(d[S], dist(i, j) + d[S^(1<<i)^(1<<j)]);
}
```

注意, 在上述的程序中求出的 i 是 S 中的最小元素, 而不是最大元素, 但这并不影响答案。另外, j 的枚举只需从 $i+1$ 开始——既然 i 是 S 中的最小元素, 则说明其他元素自然均比 i 大。最后需要说明的是 S 的枚举顺序。不难发现: 如果 S' 是 S 的真子集, 则一定有 $S' < S$, 因此若以 S 递增的顺序计算, 需要用到某个 d 值时, 它一定已经计算出来了。

提示 9-22: 如果 S' 是 S 的真子集, 则一定有 $S' < S$ 。在用递推法实现子集的动态规划时, 该规则往往可以确定计算顺序。

货郎担问题 (TSP)。 有 n 个城市, 两两之间均有道路直接相连。给出每两个城市 i 和 j 之间的道路长度 L_{ij} , 求一条经过每个城市一次且仅一次, 最后回到起点的路线, 使得经过的道路总长度最短。 $N \leq 15$, 城市编号为 $0 \sim n-1$ 。

【分析】

TSP 是一道经典的 NPC 难题^①, 不过因为本题规模小, 可以用动态规划求解。首先注意到可以直接规定起点和终点为城市 0 (想一想, 为什么), 然后设 $d(i, S)$ 表示当前在城市

^① 所谓 NPC, 即 NP-完全问题 (NP-Complete Problem), 是指一类目前还没有找到多项式算法的问题。它的确切定义超出了本书的范围。

i ，还需访问集合 S 中的城市各一次后回到城市 0 的最短长度，则

$$d(i, S) = \min\{d(j, S - \{j\}) + \text{dist}(i, j) \mid j \in S\}$$

边界为 $d(i, \{\}) = \text{dist}(0, i)$ 。最终答案是 $d(0, \{1, 2, 3, \dots, n-1\})$ ，时间复杂度为 $O(n^2 2^n)$ 。

图的色数。图论有一个经典问题是这样的：给一个无向图 G ，把图中的结点染成尽量少的颜色，使得相邻结点颜色不同。

【分析】

设 $d(S)$ 表示把结点集 S 染色，所需要颜色数的最小值，则 $d(S) = d(S - S') + 1$ ，其中 S' 是 S 的子集，并且内部没有边（即不存在 S' 内的两个结点 u 和 v 使得 u 和 v 相邻）。换句话说， S' 是一个“可以染成同一种颜色”的结点集。

首先通过预处理保存每个结点集是否可以染成同一种颜色（即“内部没有边”），则算法的主要时间取决于“高效的枚举一个集合 S 的所有子集”。

如何枚举 S 的子集呢？详见下面的代码（代码中的 S_0 就是上面的 S' ）：

```
d[0] = 0;
for(int S = 1; S < (1<<n); S++) {
    d[S] = INF;
    for(int S0 = S; S0; S0 = (S0-1)&S)
        if(no_edges_inside[S0]) d[S] = min(d[S], d[S-S0]+1);
}
```

如何分析上述算法的时间复杂度？它等于全集 $\{1, 2, \dots, n\}$ 的所有子集的“子集个数”之和。如果不好理解，可以令 $c(S)$ 表示集 S 的子集的个数（它等于 $2^{|S|}$ ），则本题的时间复杂度为 $\sum\{c(S_0) \mid S_0 \text{ 是 } \{1, 2, 3, \dots, n\} \text{ 的子集}\}$ 。元素个数相同的集合，其子集个数也相同，可以按照元素个数“合并同类项”。元素个数为 k 的集合有 $C(n, k)$ 个，其中每个集合有 2^k 个子集，因此本题的时间复杂度为 $\sum\{C(n, k)2^k\} = (2+1)^n = 3^n$ ，其中第一个等号用到了第 10 章即将学到的二项式定理（不过是“反着”用的）。

提示 9-23：枚举 $1 \sim n$ 的每个集合 S 的所有子集的总时间复杂度为 $O(3^n)$ 。

例题 9-15 校长的烦恼 (Headmaster's Headache, UVa 10817)

某校有 m 个教师和 n 个求职者，需讲授 s 个课程 ($1 \leq s \leq 8, 1 \leq m \leq 20, 1 \leq n \leq 100$)。已知每人的工资 c ($10000 \leq c \leq 50000$) 和能教的课程集合，要求支付最少的工资使得每门课都至少有两名教师能教。在职教师不能辞退。

【分析】

本题的做法有很多。一种相对容易实现的方法是：用两个集合 s_1 表示恰好有一个人教的科目集合， s_2 表示至少有两个人教的科目集合，而 $d(i, s_1, s_2)$ 表示已经考虑了前 i 个人时的最小花费。注意，把所有人一起从 0 编号，则编号 $0 \sim m-1$ 是在职教师， $m \sim m+n-1$ 是应聘者。状态转移方程为 $d(i, s_1, s_2) = \min\{d(i+1, s_1', s_2') + c[i], d(i+1, s_1, s_2)\}$ ，其中第一项表示“聘用”，第二项表示“不聘用”。当 $i \geq m$ 时状态转移方程才出现第二项。这里 s_1' 和 s_2' 分别表示“招聘第 i 个人之后 s_1 和 s_2 的新值”，具体计算方法见代码。

下面代码中的 $st[i]$ 表示第 i 个人能教的科目集合（注意输入中科目从 1 开始编号，而代

码的其他部分中科目从 0 开始编号, 因此输入时要转换一下)。下面的代码用到了一个技巧: 记忆化搜索中有一个参数 s_0 , 表示没有任何人能教的科目集合。这个参数并不需要记忆 (因为有了 s_1 和 s_2 就能算出 s_0), 仅是为了编程的方便 (详见 s_1 和 s_2 的计算方式)。最终结果是 $dp(0, (1 \leq s)-1, 0, 0)$, 因为初始时所有科目都没有人教。

```
int m, n, s, c[maxn], st[maxn], d[maxn][1<<maxs][1<<maxs];
```

```
int dp(int i, int s0, int s1, int s2) {
```

```
    if(i == m+n) return s2 == (1<<s) - 1 ? 0 : INF;
```

```
    int& ans = d[i][s1][s2];
```

```
    if(ans >= 0) return ans;
```

```
    ans = INF;
```

```
    if(i >= m) ans = dp(i+1, s0, s1, s2); //不选
```

```
    int m0 = st[i] & s0, m1 = st[i] & s1;
```

```
    s0 ^= m0; s1 = (s1 ^ m1) | m0; s2 |= m1;
```

```
    ans = min(ans, c[i] + dp(i+1, s0, s1, s2)); //选
```

```
    return ans;
```

```
}
```

本题还有其他解法, 例如, 分别用 0, 1, 2 表示每个科目是没人教、恰好一个人教和至少两个人教, 这样就可以用一个三进制数来保存状态, 而不是两个集合。不过这样做编程稍微麻烦一些, 而且时间效率差不多 (在上面的代码中, 虽然 d 数组有 4^s 个元素, 但因为记忆化的关系, 只用到了 3^s 个)。

例题 9-16 20 个问题 (Twenty Questions, ACM/ICPC Tokyo 2009, UVa1252)

有 n ($n \leq 128$) 个物体, m ($m \leq 11$) 个特征。每个物体用一个 m 位 01 串表示, 表示每个特征是具备还是不具备。我在心里想一个物体 (一定是这 n 个物体之一), 由你来猜。

你每次可以询问一个特征, 然后我会告诉你: 我心里的物体是否具备这个特征。当你确定答案之后, 就把答案告诉我 (告知答案不算“询问”)。如果你采用最优策略, 最少需要询问几次能保证猜到?

例如, 有两个物体: 1100 和 0110, 只要询问特征 1 或者特征 3, 就能保证猜到。

【分析】

为了叙述方便, 设“心里想的物体”为 W 。首先在读入时把每个物体转化为一个二进制整数。不难发现, 同一个特征不需要问两遍, 所以可以用一个集合 s 表示已经询问的特征集。在这个集合 s 中, 有些特征是 W 所具备的, 剩下的特征是 W 不具备的。用集合 a 来表示“已确认物体 W 具备的特征集”, 则 a 一定是 s 的子集。

设 $d(s, a)$ 表示已经问了特征集 s , 其中已确认 W 所具备的特征集为 a 时, 还需要询问的最小次数。如果下一次提问的对象是特征 k (这就是“决策”), 则询问次数为:

$$\max\{d(s+\{k\}, a+\{k\}), d(s+\{k\}, a)\}+1$$

考虑所有的 k , 取最小值即可。边界条件为: 如果只有一个物体满足“具备集合 a 中的所有特征, 但不具备集合 $s-a$ 中的所有特征”这一条件, 则 $d(s, a)=0$, 因为无须进一步询问,

已经可以得到答案。

因为 a 为 s 的子集, 所以状态总数为 3^m , 时间复杂度为 $O(m \cdot 3^m)$ 。对于每个 s 和 a , 可以先把满足该条件的物体个数统计出来, 保存在 $\text{cnt}[s][a]$, 避免状态转移的时候重复计算。统计 $\text{cnt}[s][a]$ 的方法是枚举 s 和物体, 时间复杂度为 $O(n \cdot 2^m)$, 所以总时间复杂度为 $O(n \cdot 2^m + m \cdot 3^m)$ 。对于本题的规模来说 $O(n \cdot 2^m)$ 可以忽略不计。

例题 9-17 基金管理 (Fund Management, ACM/ICPC NEERC 2007, UVa1412)

你有 c ($0.01 \leq c \leq 10^8$) 美元现金, 但没有股票。给你 m ($1 \leq m \leq 100$) 天时间和 n ($1 \leq n \leq 8$) 支股票供你买卖, 要求最后一天结束后不持有任何股票, 且剩余的钱最多。买股票不能赊账, 只能用现金买。

已知每只股票每天的价格 ($0.01 \sim 999.99$, 单位是美元/股) 与参数 s_i 和 k_i , 表示一手股票是 s_i ($1 \leq s_i \leq 10^6$) 股, 且每天持有的手数不能超过 k_i ($1 \leq k_i \leq k$), 其中 k 为每天持有的总手数上限。每天要么不操作, 要么选一只股票, 买或卖它的一手股票。 c 和股价均最多包含两位小数 (即美分)。最优解保证不超过 10^9 。要求输出每一天的决策 (HOLD 表示不变, SELL 表示卖, BUY 表示买)。

【分析】

根据前面的经验, 可以用 $d(i, p)$ 表示经过 i 天之后, 资产组合为 p 时的现金的最大值。其中 p 是一个 n 元组, $p_i \leq k_i$ 表示第 i 只股票有 p_i 手。根据题目规定, $p_1 + \dots + p_n \leq k$ 。因为 $0 \leq p_i \leq 8$, 理论上最多只有 $9^8 < 5 \cdot 10^7$ 种可能, 所以可以用一个九进制整数来表示 p 。

一共有 3 种决策: HOLD、BUY 和 SELL, 分别进行转移即可。注意在考虑购买股票时不要忘记判断当前拥有的现金是否足够。细心的读者可能已经发现: 正因为如此, 本题并不是一个标准的 DAG 最长/短路问题, 因为某些边 $u \rightarrow v$ 的存在性依赖于起点到 u 的最短路值。也就是说, 本题的状态不能像之前的 DAG 问题一样“反着定义”: 如果用 $d(i, p)$ 表示资产组合为 p , 从第 i 天开始到最后能拥有的现金的最大值, 就没法转移了 (想一想, 为什么)。

这样的做法虽然不错^①, 但是效率却不够高, 因为九进制整数无法直接进行“买卖股票”的操作, 需要解码成 n 元组才行。因为几乎每次状态转移都会涉及编码、解码操作, 状态转移的时间大幅度提升, 最终导致超时。

解决方法是事先计算出所有可能的状态并且编号 (还记得第 5 章中的“集合栈计算机”吗?), 代码如下:

```
vector<vector<int>> > states;
map<vector<int>, int> ID;

void dfs(int stock, vector<int>& lots, int totlot) {
    if (stock == n) {
        ID[lots] = states.size();
        states.push_back(lots);
    }
    else for (int i = 0; i <= k[stock] && totlot + i <= kk; i++) {
```

^① 完整实现见代码仓库。

```

    lots[stock] = i;
    dfs(stock+1, lots, totlot + i);
}
}

```

然后构造一个状态转移表，用 `buy_next[s][i]` 和 `sell_next[s][i]` 分别表示状态 `s` 进行“买股票 `i`”和“卖股票 `i`”之后转移到的状态编号，代码如下：

```
int buy_next[maxstate][maxn], sell_next[maxstate][maxn];
```

```
void init() {
```

```
    vector<int> lots(n);
```

```
    states.clear();
```

```
    ID.clear();
```

```
    dfs(0, lots, 0);
```

```
    for(int s = 0; s < states.size(); s++) {
```

```
        int totlot = 0;
```

```
        for(int i = 0; i < n; i++) totlot += states[s][i];
```

```
        for(int i = 0; i < n; i++) {
```

```
            buy_next[s][i] = sell_next[s][i] = -1;
```

```
            if(states[s][i] < k[i] && totlot < kk) {
```

```
                vector<int> newstate = states[s];
```

```
                newstate[i]++;
```

```
                buy_next[s][i] = ID[newstate];
```

```
            }
```

```
            if(states[s][i] > 0) {
```

```
                vector<int> newstate = states[s];
```

```
                newstate[i]--;
```

```
                sell_next[s][i] = ID[newstate];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

动态规划主程序采用刷表法（读者也可以试着改成倒推的填表法），为了方便起见，另外编写了“更新状态”的函数 `update`，读者可以自行体会它的好处。为了打印解，在更新解 `d` 时还要更新最优策略 `opt` 和“上一个状态” `prev`。注意下面的 `price[i][day]` 表示第 `day` 天时一手股票 `i` 的价格，而不是输入中的“每股价格”。

```
double d[maxm][maxstate];
```

```
int opt[maxm][maxstate], prev[maxm][maxstate];
```

```
void update(int day, int s, int s2, double v, int o) {
```

```
    if(v > d[day+1][s2]) {
```

```

    d[day+1][s2] = v;
    opt[day+1][s2] = o;
    prev[day+1][s2] = s;
}
}

double dp() {
    for(int day = 0; day <= m; day++)
        for(int s = 0; s < states.size(); s++) d[day][s] = -INF;
    d[0][0] = c;
    for(int day = 0; day < m; day++)
        for(int s = 0; s < states.size(); s++) {
            double v = d[day][s];
            if(v < -1) continue;
            update(day, s, s, v, 0); //HOLD
            for(int i = 0; i < n; i++) {
                if(buy_next[s][i] >= 0 && v >= price[i][day] - 1e-3)
                    update(day, s, buy_next[s][i], v - price[i][day], i+1); //BUY
                if(sell_next[s][i] >= 0)
                    update(day, s, sell_next[s][i], v + price[i][day], -i-1); //SELL
            }
        }
    return d[m][0];
}

```

最后是打印解的部分。因为状态从前到后定义，因此打印解时需要从后到前打印，用递归比较方便。

```

void print_ans(int day, int s) {
    if(day == 0) return;
    print_ans(day-1, prev[day][s]);
    if(opt[day][s] == 0) printf("HOLD\n");
    else if(opt[day][s] > 0) printf("BUY %s\n", name[opt[day][s]-1]);
    else printf("SELL %s\n", name[-opt[day][s]-1]);
}

```

9.5 竞赛题目选讲

例题 9-18 跳舞机 (Tango Tango Insurrection, UVa 10618)

你想学着玩跳舞机。跳舞机的踏板上有 4 个箭头：上、下、左、右。当舞曲开始时，屏幕上会有一些箭头往上移动。当向上移动箭头与顶部的箭头模板重合时，你需要用脚踩

一下踏板上的相同箭头。不需要踩箭头时，踩箭头并不会受到惩罚，但当需要踩箭头时，必须踩一下，哪怕已经有一只脚放在了该箭头上。很多舞曲的速度快，需要来回倒腾步子，所以最好写一个程序来帮助你选择一个轻松的踩踏方式，使得能量消耗最少。

为了简单起见，将一个八分音符作为一个基本时间单位，每个时间单位要么需要踩一个箭头（不会同时需要踩两个箭头），要么什么都不需要踩。在任意时刻，你的左右脚应放在不同的两个箭头上，且每个时间单位内只有一只脚能动（移动和/或踩箭头），不能跳跃。另外，你必须面朝前方以看到屏幕（例如，你不能把左脚放到右箭头上，右脚放到左箭头上）。

当你执行一个动作（移动和/或踩）时，消耗的能量这样计算：

- 如果这只脚上个时间单位没有任何动作，消耗 1 单位能量。
- 如果这只脚上个时间单位没有移动，消耗 3 单位能量。
- 如果这只脚上个时间单位移动到相邻箭头，消耗 5 单位能量。
- 如果这只脚上个时间单位移动到相对箭头（上到下，或者左到右），消耗 7 单位能量。

正常情况下，你的左脚不能放到右箭头上（或者反之），但有一种情况例外：如果你的左脚在上箭头或者下箭头，你可以临时扭着身子用右脚踩左箭头，但是在你的右脚移出左箭头之前，你的左脚都不能移到另一个箭头上。类似地，右脚在上箭头或者下箭头时，你也可以临时用左脚踩右箭头。一开始，你的左脚在左箭头上，右脚在右箭头上。

输入包含最多 100 组数据，每组数据包含一个长度不超过 70 的字符串，即各个时间单位需要踩的箭头。L 和 R 分别表示左右箭头，“.”表示不需要踩箭头。输出应是一个长度和输入相同的字符串，表示每个时间单位执行动作的脚。L 和 R 分别是左右脚，“.”表示不踩。

【分析】

虽然本题的条件比较杂乱，但总的来说不难发现：可以按“箭头”划分阶段，再记录一下左右脚的位置以及上次左脚有没有踩，就可以顺利地动态规划了。

具体来说，用 $d(i, a, b, s)$ 表示已经踩了 i 个箭头 ($i \geq 0$)，左右脚分别在箭头 a 和 b 上，且上一个周期移动脚的集合为 s ($s=0$ 表示没有脚移动， $s=1$ 表示左脚移动， $s=2$ 表示右脚移动)，则最终答案为 $d(0, 1, 2, 0)$ 。4 个箭头的编号为 0-上，1-左，2-右，3-下。

如果下一步是“.”，有 3 种决策：左脚移动到另一个箭头；右脚移动到另一个箭头；不动。注意，虽然这次移动什么箭头都不会踩到，但还是要输出移动脚。

如果下一步是 4 个箭头之一，有两种决策：左脚移动到该箭头；右脚移动到该箭头。注意不要枚举不符合题目要求的移动方式。

例题 9-19 团队分组 (Team them up!, ACM/ICPC NEERC 2001, UVa1627)

有 n ($n \leq 100$) 个人，把他们分成非空的两组，使得每个人都分到一组，且同组中的人相互认识。要求两组的成员人数尽量接近。多解时输出任意方案，无解时输出 No Solution。

例如，1 认识 2, 3, 5；2 认识 1, 3, 4, 5；3 认识 1, 2, 5，4 认识 1, 2, 3，5 认识 1, 2, 3, 4（注意 4 认识 1 但 1 不认识 4），则可以分两组： $\{1, 3, 5\}$ 和 $\{2, 4\}$ 。

【分析】

设两个组的编号为 0 和 1。因为同组中的人相互认识，所以如果有两个人 a 和 b 不是相互认识，那么 a 和 b 只能分到两个不同的组。这样，如果已知某个人是第 0 组，那么不认

识它的所有人都应该是第1组。而不认识这些人的所有人都应该是0组，依此类推。这样，如果把“不相互认识”关系看成一个图，则每个连通分量都可以独立推导（推导过程中可能遇到矛盾，此时原问题无解）。例如，上面的样例对应图9-16（注意a认识b，但b不认识a，也应该连一条边）。

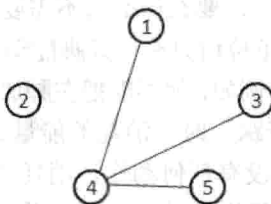


图 9-16 团队分组样例示意图

对于连通分量{1,3,4,5}，假设1在组0，可以推导出3,4,5都在组1；反过来，如果1在组1，可以推导出3,4,5都在组0。设组0比组1的人数多d个，可以总结出如表9-1所示。

表 9-1 组0和组1人数分布

	情况 1	情况 2
连通分量 1	组 0: {2}; 组 1: {} (d 加 1)	组 0: {}; 组 1: {2} (d 减 1)
连通分量 2	组 0: {4}; 组 1: {1,3,5} (d 减 2)	组 0: {1,3,5}; 组 1: {4} (d 加 2)

可以看到，每个连通分量的两种情况分别对应于 d 加一个值或者减一个值，最终目标是 d 的绝对值尽量少。想到了什么？没错！是 0-1 背包问题，只是没有“体积”，而“重量”有正有负，最后也不是要“重量”最大，而是最接近 0。

例题 9-20 装满水的气球（Dropping water balloons, UVa 10934）

一年一度的新生周活动开始了，你们做好了大量的装满水的气球，准备拿来恶搞那些可怜的新生。活动开始之前，你们突然发现一个问题：这些气球实在是太硬了，很难把它们打破（如果打不破，它们就没有任何意义了）。甚至从好几层高的楼顶上把它们扔到地面，也打不破。你的任务是借助一个 n 层的高楼确定气球的硬度（所有气球硬度相同）。

实验过程是这样的：每次你拿着一个气球爬到第 f 层楼，将它摔到地面。如果气球破了，说明它的硬度不超过 f；如果没破，说明硬度至少为 f。注意，气球不会被实验所“磨损”。换句话说，如果在某层楼上往下摔，气球没破，那么在同一层楼不管再摔多少次它也不会破。

给你 k 个气球用来实验（可以打破它们）。你的任务是求出至少需要多少次实验，才能确定气球的硬度（或者得出结论：站在最高层也摔不破）。

输入每行包含两个整数 k, n ($1 \leq k \leq 100$, $1 \leq n < 2^{64}$)，输出最少需要的实验次数。如果 63 次不够，输出“More than 63 trials needed”。

【分析】

用状态 $d(i, j)$ 表示用 i 个球实验 j 次所能测试的楼的最高层数。根据动态规划的常见思路，我们考虑第一次决策，设测试楼层为 k。

如果气球破了，说明前 k-1 层必须能用 i-1 个球实验 j-1 次测出来，也就是说，取 $k=d(i-1, j-1)+1$ 是最优的。

如果气球没有破,则相当于把第 $k+1$ 层楼看作 1 楼以后继续。因此在第 k 层楼之上还可以测 $d(i,j-1)$ 层楼,即 $d(i,j) = k + d(i,j-1) = d(i-1,j-1) + 1 + d(i,j-1)$ 。

例题 9-21 修缮长城 (Fixing the Great Wall, ACM/ICPC CERC 2004, UVa1336)

长城被看作一条直线段,有 n ($1 \leq n \leq 1000$) 个损坏点需要用机器人 GWARR 修缮。可以用三元组 (x_i, c_i, d_i) 描述第 i 个损坏点的参数,其中 x_i 是位置, c_i 是立刻修缮 (即时刻=0 时开始修缮) 的费用, d_i 是单位时间增加的修缮费用。换句话说,如果在时刻 t_i 开始修缮第 i 个损坏点,费用为 $c_i + t_i d_i$ 。上述参数满足 $1 \leq x_i \leq 500000$, $0 \leq c_i \leq 50000$, $1 \leq d_i \leq 50000$ 。

修缮的时间忽略不计, GWARR 的速度恒定为 v ($1 \leq v \leq 100$), 因此从修缮点 i 走到修缮点 j 需要 $|x_i - x_j|/v$ 单位的时间。初始坐标为 x ($1 \leq x \leq 500000$)。输入保证损坏点的位置各不相同,且 GWARR 的初始位置不与任何一个损坏点重合。

你的任务是找到修缮所有点的最小费用 (用截尾法保留整数部分)。输入保证最小费用不超过 10^9 。

【分析】

首先将所有修缮点按照坐标从小到大排序,不难发现在任意时候,已修复的点一定是一个连续的区间,因此可以考虑用 $d(i,j,k)$ 表示修复完 (i,j) ,且当前位置为 k ($k=0$ 表示在左端点 i , $k=1$ 表示在右端点 j) 时已经发生的总费用。

但是这样会带来一个问题:今后的费用无法计算,因为不知道当前时间。不过没关系,谁说必须当费用发生以后才能计算?可以事先把还没有发生但是肯定会发生的费用累加到答案中,然后“时钟归零”。事实上,在前面已经用过一次这种技巧了,那就是例题“颜色的长度”。

设 $d(i,j,k)$ 表示修复完 (i,j) ,且当前位置为 k (含义同上) 时,已经发生的总费用与所有“肯定会发生的未来费用”之和,使用刷表法,则一共只有两个决策。

决策 1: 往左走,修理点 $i-1$, 转移到 $d(i-1,j,0)$ 。假设当前点为 p ($k=0$ 时 $p=i$, 否则 $p=j$), 则到达点 $i-1$ 的时间为 $t = |X_{i-1} - X_p|/v$ 。在这段时间里,所有未修理点 (即点 $1 \sim i-1$ 和 $j+1 \sim n$) 的费用都增加了 t ,需要把这些点的总费用 $(\text{sum_d}(1,i-1) + \text{sum_d}(j+1,n)) * t$ 累加到状态值中,然后点 $i-1$ 的修理费用就只有 c_{i-1} 了。即用 $d(i,j,k) + (\text{sum_d}(1,i-1) + \text{sum_d}(j+1,n)) * t + c_{i-1}$ 来更新 $d(i-1,j,0)$ 。其中 $\text{sum_d}(i,j)$ 表示点 $i \sim j$ 的所有 d 值之和。

决策 2: 往右走,修理点 $j+1$, 转移到 $d(i,j+1,1)$ 。和决策 1 很类似,方程略。

状态有 $O(n^2)$ 个,每个状态只有两个决策,因此时间复杂度为 $O(n^2)$ 。

例题 9-22 越大越好 (Bigger is Better, ACM/ICPC Xi'an 2006, UVa12105)

你的任务是用不超过 n ($n \leq 100$) 根火柴摆一个尽量大的,能被 m ($m \leq 3000$) 整除的正整数。例如, $n=6$ 和 $m=3$, 解为 666。无解输出 -1, 如图 9-17 所示。

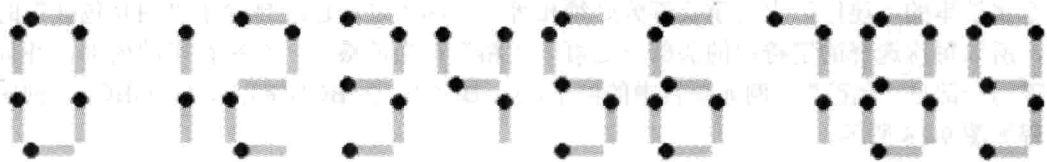


图 9-17 火柴数字

【分析】

一般来说，整数是从左往右一位一位写的，因此不难想到这样的动态规划算法：用 $d(i, j)$ 表示用 i 根火柴能拼出的“除以 m 余数为 j ”的最大数，然后用刷表法，枚举在最右边添加的数字 k ，用 $d(i, j) * 10 + k$ 更新 $d(i + c(k), (j * 10 + k) \% m)$ ，其中 $c(k)$ 表示数字 k 需要的火柴数。状态有 $O(nm)$ 个，每个状态只有“在右边添加数字 0~9”这 10 个决策，看上去不错。可惜这个算法有个缺点：状态值是高精度整数，因此实际计算量比较大。

还有一个算法，虽然有些难想，但是效率很高：用 $d(i, j)$ 表示拼出一个“除以 m 余数为 j 的 i 位数”至少需要多少火柴（若无解， $d(i, j)$ 为正无穷）。状态转移方程和上面类似，留给读者思考。因为此处只关心位数，这个算法并不涉及高精度整数。

如何根据 $d(i, j)$ 计算出题目要求的答案呢？首先确定最大的位数 w （即让 $d(i, 0)$ 不是正无穷的最大 i ），因为位数越大，整数就越大（不允许有前导 0，因为不划算）。接下来从左到右依次确定各个数字。

例如，假定 $m=7$ ，并且已经确定最大的整数是 3 位数。首先试着让最高位为 9。如果可以摆出形如 9ab 的整数，它一定是最大的。是否可以摆出 9ab 呢？因为 900 除以 7 的余数为 4，后两位“ab”除以 7 的余数应为 3。如果 $d(2, 3) + c(9) \leq n$ ，说明火柴足够摆出 9ab，否则说明最高位不能是 9。重复这个过程，直到所有数字都被确定为止。这个过程需要快速算出形如 $x000\cdots$ 的整数除以 m 的余数，可以通过一个预处理完成，留给读者思考。

例题 9-23 有趣的游戏 (Fun Game, ACM/ICPC Beijing 2004, UVa1204)

一些小孩围成一圈做游戏。每一轮从某个小孩开始往他左边或右边传手帕。一个小孩拿到手帕后（包括第一个小孩）在手帕上写下自己的性别，男孩写 B，女孩写 G，然后按相同方向传给下一个小孩，每一轮可能在任何一个小孩写完后停止。现在游戏已经进行了 n 轮，已知 n 轮中每轮手帕上留下的字，求最少可能有多少小孩。 $2 \leq n \leq 16$ 。每轮手帕上的字数不超过 100。

例如，若 3 轮的手帕上分别留下 BGGB, BGBGG, GGGBGB，则至少有 9 个小孩。一种可能性是 GGGBGBGB。

【分析】

首先可以看出，如果有一个字符串完全包含于其他某个字符串，那么这个字符串将对结果没有影响，所以先预处理去掉这些字符串。后面将看到这会给动态规划带来方便。

在解决原题之前，先看一个简化版：小孩排成一行（而不是一圈），且传递手帕总是从左到右的。那么问题就等价于：找一个最短的字符串，使得输入的 n 个字符串都是它的连续子串。

可以把这个问题转化为一个多阶段决策过程：每次选择一个字符串“粘”在当前最后一个字符串的“尾巴”上（重叠部分必须相等）。因为之前已经排除了“相互包含”的情况，所以每次选择的字符串的头部一定可以“粘”在当前最后一个字符串的内部，并且可以露出一部分“尾巴”。例如题目中的例子， $s_1 = \text{BGGB}$, $s_2 = \text{BGBGG}$, $s_3 = \text{GGGBGB}$ ，则决策过程如图 9-18 所示。

最终得到的字符串长度等于所有 n 个字符串的长度之和，减去每个串（除了第一个串）与前一个串的最大重叠长度。对于上面的例子， s_1, s_2, s_3 的长度之和为 15， s_2 和 s_3 的最大

重叠长度为 3, s_1 和 s_2 的最大重叠长度为 3, 因此最终得到的字符串长度为 $15-3-3=9$ 。注意上述“最大重叠长度”不是对称的, 例如, 若 s_2 在右边, s_2 和 s_3 可以重叠 3 个字符, 但如果 s_2 在左边, 则只能重叠 2 个字符。

这个过程启发我们使用动态规划。用 $d(i, j)$ 来表示已经选过的字符串集合为 i , 最后一个串为 j 时, 可以减去的重叠部分总长。如图 9-19 所示, 假设已经选择了字符串 1, 6, 4, 其中最后一个字符串为 4, 即状态 $d(\{1, 4, 6\}, 4)$ 。假设接下来选择字符串 3, 并且已经得到了 3 粘在 4 尾巴上时的最大重叠长度为 5, 则可以用 $d(\{1, 4, 6\}, 4)+5$ 来更新 $d(\{1, 3, 4, 6\}, 3)$ 。

GGGBGB s_3
BGBGG s_2
BGGB s_1

图 9-18 决策过程

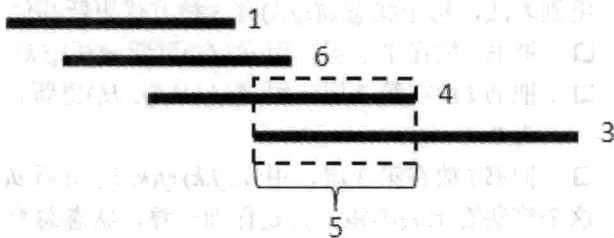


图 9-19 已选字符串 1, 6, 4 的情况

现在已经解决了简化版问题, 原题只有两点不同:

(1) 原题中, 手帕有两种不同的方向, 因此选择每个串之后, 还要确定是把它直接粘上呢, 还是反过来粘, 因此状态 $d(i, j)$ 中的 j 有 $2n$ 种可能, 每次的决策也变成 $2n$ 个, 时间复杂度不变, 只是常数略有增加。

(2) 原题中, 所有小孩组成一个圈, 因此需要考虑如何把链变成圈。一种方法是在状态中增加一维, 用来记录第一个串是哪个, 这样就可以在最后一次决策时计算最后一个串和第一个串的公共部分。这样做并没有错, 但是因为状态多了一维, 时间复杂度也将变大。其实, 第一个串和最后一个串并不一定会重叠。如果是那样, 按链的方式计算出的就是正确答案; 如果第一个串和最后一个串重叠, 但是中间某个串 i 和下一个串不重叠, 链的答案仍然是正确答案, 因为可以在串 i 后面把圈“打断”成链, 让 i 成为“最后一个串”。如果每个串和后一个串都有重叠怎么办? 那样只能按照圈的做法了, 不过任选一个串作为第一个串都行, 不用给状态增加一维。另外还有一个地方要注意: 输入字符串不一定是圈的一部分, 它可能绕了好几圈(想一想, 应如何处理)。

这样, 即把简化版问题的解扩展成了原题的解法, 时间复杂度仍是 $O(n^2 \cdot 2^n)$ 。

例题 9-24 书架 (Bookcase, ACM/ICPC NWERC 2006, UVa12099)

有 n ($3 \leq n \leq 70$) 本书, 每本书有一个高度 H_i 和宽度 W_i ($150 \leq H_i \leq 300$, $5 \leq W_i \leq 30$)。现在要构建一个三层的书架, 你可以选择将 n 本书放在书架的哪一层。设三层高度 (该层书的最大高度) 之和为 h , 书架总宽度 (即每层总宽度的最大值) 为 w , 则要求 $h \cdot w$ 尽量小。

【分析】

如果所有书的高度都相等, 本题就是“分成 3 个子集, 使得元素和的最大值尽量小”, 而这是 0-1 背包类型的问题。这提示我们需要把宽度写到状态里。

首先将所有的书按照高度从大到小排序。不妨设高度最大的书安排在第 1 层, 且第 2 层的高度大于等于第 3 层的高度, 然后设状态 $d(i, j, k)$ 表示安排完前 i 本书, 第 2 层书的宽度

之和为 j ，第 3 层书的宽度之和为 k 时，第 2 层高度和第 3 层高度和的最小值。

为什么不记录第 1 层的高度？因为最高的书在第 1 层，意味着这一层永远都不会比它更高了；为什么不记录第 1 层的宽度？因为目前 3 层的总宽度等于前 i 本书的总宽度，只要知道了第 2、3 层的宽度，就能算出第 1 层的宽度。另外，因为这些书已经按照高度从大到小排序了，一旦 3 层都放了书，3 层的高度都不会变了，因此：

- 如果只有前两层放了书，当且仅当往第 3 层放书 i 时，第 3 层高度会从 0 变到 H_i 。
 - 如果只有第 1 层放了书，当且仅当往第 2 层放书 i 时，第 2 层高度会从 0 变到 H_i 。
- 用刷表法，每个状态 $d(i, j, k)$ 有 3 种方式更新其他状态：
- 把书 i 放在第 1 层，用 $d(i, j, k)$ 更新 $d(i+1, j, k)$ ，因为第 1 层高度不变。
 - 把书 i 放在第 2 层，用 $d(i, j, k) + f(j, H_i)$ 更新 $d(i+1, j+W_i, k)$ ，其中 $f(0, h) = h$ ，其他 f 值为 0。
 - 把书 i 放在第 3 层，用 $d(i, j, k) + f(k, H_i)$ 更新 $d(i+1, j, k+W_i)$ ， f 函数的定义同上。

这个算法看上去不错，但是仔细一算，状态总数为 $70 * 2100 * 2100$ ，太大了——就算作用时间能接受，所占用的空间也无法接受，因此无法使用记忆化搜索，而只能用递推，配合滚动数组（由于是 0-1 背包式的递推， i 那一维可以完全省略）。

如何优化呢？出乎大多数选手的意料^①，本题的“标准优化”并没有降低理论时间复杂度，只是让程序的实际运行效率高了很多。优化有两种：

- $j+k$ 不应该超过前 i 本书的宽度之和，因此有用的状态比 $70 * 2100 * 2100$ 少得多。
- 假设第 i 层书的总宽度为 ww_i ，如果 $ww_2 > ww_1 + 30$ （30 是一本书的宽度上限），那么可以把第 2 层的一本书放到第 1 层来，则前两层高度之和不会变大，书架宽度（即两层总宽度的最大值）也不会变大。因此，只需要计算满足 $ww_2 \leq ww_1 + 30$ 且 $ww_3 \leq ww_2 + 30$ 的状态，因此 $j \leq (2100 + 30) / 2 = 1065$ ， $k \leq (2100 + 60) / 3 = 720$ 。

强烈建议读者实现优化前后的两个版本，比较二者的效果。

例题 9-25 轻松爬山 (Easy Climb, NWERC 2008, UVa12170)

输入正整数 d 和 n 个正整数 $h_1, h_2, \dots, h_n$ ，可以修改除了 h_1 和 h_n 的其他数，要求修改后相邻两个数之差的绝对值不超过 d ，且修改费用最小。设 h_i 修改之后的值为 h'_i ，则修改费用为 $|h_1 - h'_1| + |h_2 - h'_2| + \dots + |h_n - h'_n|$ 。无解输出 -1。 $N \leq 100$ ， $d \leq 10^9$ 。

【分析】

本题是一个多阶段决策过程：依次确定每个 h_i 修改成什么数。可惜 d 的范围太大，如果用 $f(i, x)$ 表示已经修改 i 个数，其中第 i 个数改成 x 时还需要的最小费用，则状态总数高达 $O(nd)$ 。

为了更好地分析问题，先来看看简化版： $n=3$ 时，只有 h_2 是可以修改的，而且修改之后必须同时在 $[h_1-d, h_1+d]$ 和 $[h_3-d, h_3+d]$ 内，即 $[\max(h_1, h_3)-d, \min(h_1, h_3)+d]$ 。如果这个区间是空的，说明无解；否则 h_2 要么不变，要么改成 $\max(h_1, h_3)-d$ 或者 $\min(h_1, h_3)+d$ 。

这个例子至少说明了：修改后的值并不是随便选的，至少在 $n=3$ 时，修改后的值只有 3 种选择： h_2 ， $\max(h_1, h_3)-d$ 和 $\min(h_1, h_3)+d$ 。

^① 本题的解法看上去比较常规，但是在 NWERC 这样较高水平的比赛中，却没有队伍做出来。

用类似的推理,可以得到这样的结论:每个数在修改之后一定可以写成 $h_p + kd$, 其中 $1 \leq p \leq n$, $-n < k < n$, 这样, 上述状态 $f(i, x)$ 中的 “ x ” 就只有 $O(n^2)$ 种可能了, 状态总数为 $O(n^3)$ 。

不难写出状态转移方程: $f(i, x) = |h_i - x| + \min \{f(i-1, y) \mid x-d \leq y \leq x+d\}$ 。如果按照 x 从小到大的顺序计算, 满足 $x-d \leq y \leq x+d$ 的 $f(i-1, y)$ 就是 $i-1$ 阶段状态值序列的一个滑动窗口。使用前面介绍过的单调队列, 可以在平摊 $O(1)$ 的时间复杂度内计算出 $f(i, x)$, 因此本题的总时间复杂度为 $O(n^3)$ 。

例题 9-26 一个调度问题 (A Scheduling Problem, ACM/ICPC Kaoshiung 2006, UVa1380)

有 n ($n \leq 200$) 个恰好需要一天完成的任务, 要求用最少的时间完成所有任务。任务可以并行完成, 但必须满足一些约束。约束分有向和无向两种, 其中 $A \rightarrow B$ 表示 A 必须在 B 之前完成, $A-B$ 表示 A 和 B 不能在同一天完成。输入保证约束图是将一棵 n ($n \leq 200$) 个结点的树的某些边定向后得到的。例如, 图 9-20 表示 1 和 2 不能在同一天完成, 1 必须在 3 之前, 3 必须在 5 之前, 2 必须在 4 之前, 4 必须在 6 之前。

可以使用如下定理: 忽略无向边之后, 设图上的最长链 (即包含点数最多的路径) 包含 k 个点, 则答案为 k 或者 $k+1$ 。对于上面的例子, 忽略无向边后的最长链是 $2 \rightarrow 4 \rightarrow 6$, 包含 3 个结点。

【分析】

如果树中所有边都为有向边, 那么答案就是最长链上的点数: 先将度为 0 的点全部安排在第一, 将这些点删去, 然后将新的度为 0 的点安排在第二天, 这样就可以在 k 天内安排完。这样, 原问题转化为: 将树中的所有无向边定向, 使得树中的最长链最短。

根据题目中的定理, 设原图中有向边组成的最长链上有 k 个点, 那么最终的答案不是 k 就是 $k+1$, 接下来只需要判断是否可以通过无向边定向使得最长链的点数为 k 。即使没有题目中的那个定理, 也可以二分答案 x , 然后判断是否能让最长链的点不超过 x 。不管是哪种情况, 问题的关键就是: 给定一个 x , 判断是否可以给无向边定向, 使得最长链点数不超过 x 。

设 $f(i)$ 表示以 i 为根的子树内的边全部定向后, 最长链点数不超过 x 的前提下, 形如 “后代到 i ” (如图 9-21 中的 $u' \rightarrow u \rightarrow i$) 的最长链的最小值, 同理可以定义 $g(i)$ 表示形如 “ i 到后代” (如图 9-21 中的 $i \rightarrow v \rightarrow v'$) 的最长链的最小值。达不到的状态 (即 “最长链点数不超过 x ” 这个前提无法满足) 定义为正无穷。

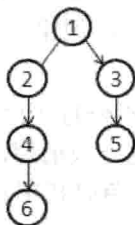


图 9-20 调度问题示意图

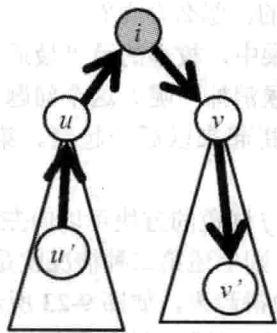


图 9-21 最长链

如何计算 $f(i)$ 和 $g(i)$ 呢？为了叙述方便，用 w 表示 i 的某个子结点，则 w 和 i 之间的边有 3 种情况： $w \rightarrow i$ ， $i \rightarrow w$ 和 $i-w$ ，其中前两种是有向边，最后一种是无向边。按照从易到难的顺序，分两种情况讨论。

情况 1：如果 i 与 w 的所有边都是有向边，直接计算即可。令 $f(i)$ 等于形如 $w \rightarrow i$ 的 w 的 $f(w)$ 的最大值加 1， $g'(i)$ 等于形如 $i \rightarrow w$ 的 w 的 $g(w)$ 的最大值加 1，则以 i 为根的子树内，经过 i 的最长链点数等于 $f(i)+g'(i)$ 。如果这个值大于 x ，则 $f(i)$ 和 $g(i)$ 为正无穷，否则 $f(i)=f(i)$ ， $g(i)=g'(i)$ 。

情况 2：如果 i 与某些 w 之间存在无向边，则需要确定每条 $i-w$ 定向成 $w \rightarrow i$ 还是 $i \rightarrow w$ 。由于定向完成之后，仍需要按照情况 1 的方法计算，所以问题的关键是分析“定向”操作会如何影响 $f(i)$ 和 $g'(i)$ 。

求 $f(i)$ 时，目标是 $f(i)+g'(i) \leq x$ 的前提下 $f(i)$ 最小。首先把所有没定向的 $f(w)$ 从小到大排序。假定把 f 值第 p 小的 w 定向为 $w \rightarrow i$ ，那么最好“顺便”把前 p 小的全部变成 $w \rightarrow i$ 的，因为这样做不会让 $f(i)$ 变大，但有可能让 $g'(i)$ 变小，百利而无一害。所以只需要枚举 p ，把 f 值前 p 小的 w 都定向为 $w \rightarrow i$ ，其他定向为 $i \rightarrow w$ ，然后计算 $f(i)$ 。用相同的方法可以计算 $g(i)$ 。最后判断根结点的 f 值是否无穷大即可。

例题 9-27 方块消除 (Blocks, UVa10559)

有 n ($n \leq 200$) 个带颜色方格排成一列，相同颜色的方块连成一个区域。游戏时，可以任选一个区域消去。设这个区域包含的方块数为 x ，则将得到 x^2 个分值，然后右边所有方块就会向左移一格。如图 9-22 所示是一个游戏局面和最优消除方式。

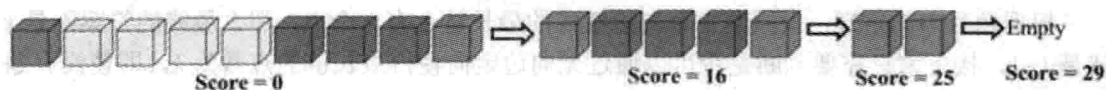


图 9-22 游戏局面和最优消除方式

你的任务是求出最高可能的得分。

【分析】

为了叙述方便，设左数第 i 个方块的颜色为 $A[i]$ 。按照线性结构动态规划的常见思路，设 $d(i,j)$ 表示子序列 $i \sim j$ 的最大得分，但是似乎无法用 $d(i,k)$ 和 $d(k,j)$ 来计算 $d(i,j)$ ，因为可能 $i \sim k$ 和 $k \sim j$ 各剩下一些，拼起来以后消除。如 XAXBXCXDXEX，实际上是把 A 和 E 全部单个消除以后再消除 X 的。怎么办呢？

在最优矩阵链乘中，枚举的是“最后一次乘法”的位置。本题是不是也可以枚举“最后一个方块什么时候消掉”呢？这个问题的答案有两种可能：直接把它所在的一段消掉；把它和左边的某段拼起来以后一起消。第一种情况容易处理，但第二种情况就没那么简单了。

具体来说，设与 j 同色的方块可以向左延伸到 p (即 $A[p]=A[p+1]=\dots=A[j]$)，且 $A[q]=A[j]$ ， $A[q]$ 不等于 $A[q+1]$ ，则上述第二种情况就是指先把 $q+1 \sim p-1$ 这一段消掉，把 $p \sim j$ 这一段和以 q 为右端点的那一段拼起来，如图 9-23 所示。注意 $i \sim j$ 全部同色时找不到这样的 q ，但此时可以直接计算出结果。下面忽略这种情况。

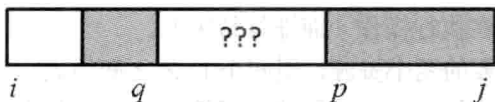


图 9-23 消掉与拼接方块

不过,把这两段拼起来以后仍然不一定立刻消除,还可能要和更左边的另一段拼起来……是不是很复杂?但有一点是可以肯定的,那就是 $q+1 \sim p-1$ 这一段肯定可以先消掉(拖到后面再消也得不到什么好处)。那么现在就把它消掉(得分是 $d(q+1, p-1)$),得到一个“子序列 $i \sim q$ 的右边再拼上 $j-p+1$ 个与 $A[q]$ 同色的方块”的奇怪状态,如图 9-24 所示。

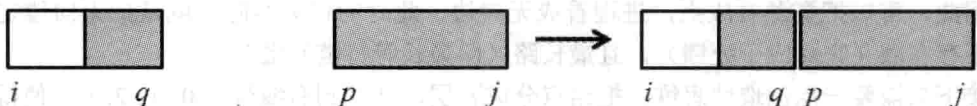


图 9-24 消掉后的奇怪状态

由此可知,在状态中增加一维,来表达“右边拼上一些方块”,即用 $d(i, j, k)$ 表示“原序列中的方块 $i \sim j$ 右边再拼上 k 个颜色等于 $A[j]$ 的方块所得到的新序列”的最大得分,则决策有两种。

决策 1: 直接消去方块 j , 转移到 $d(i, p-1, 0) + (j-p+1)^2$ 。

决策 2: 枚举 $q < p$ 使得 $A[q] = A[j]$ 且 $A[q] \neq A[q+1]$, 转移到 $d(q+1, p-1, 0) + d(i, q, j-p+1)$ 。

状态有 $O(n^3)$ 个, 决策有 $O(n)$ 个, 时间复杂度为 $O(n^4)$ 。如果采用记忆化搜索, 很多状态都达不到, 而且 q 的取值范围往往很小, 所以对于大部分数据, 这个算法的运行效率都很高。

例题 9-28 独占访问 2 (Exclusive Access 2, ACM/ICPC NEERC 2009, UVa1439)

在一个庞大的系统里运行着 n ($1 \leq n \leq 100$) 个守护进程。每个进程恰好用到两个资源。这些资源不支持并发访问, 所以这些进程通过锁来保证互斥访问。每个进程的主循环如下:

```
loop forever
  DoSomeNonCriticalWork()
  P.lock()
  Q.lock()
  WorkWithResourcesPandQ()
  Q.unlock()
  P.unlock()
end loop
```

注意, P 和 Q 的顺序是至关重要的。如果某进程用到了消息队列和数据库, “先获取数据库的锁”与“先获取消息队列的锁”可能会产生截然不同的效果。给定每个进程所需要的两种资源, 你的任务是确定每个进程获取锁的顺序, 使得进程永远不会死锁, 且最坏情况下, 等待链的最大长度最短。

在本题中, 一个长度为 n 的等待链是一个不同资源和不同进程的交替序列: $R_0 c_0 R_1 c_1 \dots R_n c_n R_{n+1}$, 其中进程 c_i 已经获取 R_i 的锁, 正在等待 R_{i+1} 的锁。当 $R_0 = R_{n+1}$ 时死锁, 否则说明

已获取 R_{n+1} 的锁的进程正在执行操作（而非等待中）。

输入 n 和每个进程需要的两个资源，用两个 $L \sim Z$ 之间的大写字符表示（因此一共有 15 种资源）。输出包含两行，第一行为最坏情况下等待链的最大长度 m ，以下 n 行每行输出两个字符，表示该进程获取锁的顺序（先获取第一个字符对应资源的锁）。

【分析】

本题初看起来毫无头绪，甚至连数学模型都难以建立。注意，每个进程恰好需要两个资源，而等待链的定义是资源和进程的交替序列，可以联想到图论中的概念：每条边恰好连接两个点，路径的定义是点和边的交替序列。

因此，可以把资源看成点，进程看成无向边，此时的任务实际上就是把无向边定向，使得不存在圈（它对应于死锁），且最长路（即最长等待链）最短。

接下来需要一点创造性思维：把结点分成 p 层，从左到右编号为 $0, 1, 2, \dots$ ，使得同层结点之间没有边。对于任意一条边 $u-v$ ，把它定向成“从层编号小的点指向层编号大的点”。例如，若 u 在第 5 层， v 在第 2 层，则定向为 $v \rightarrow u$ 。定向之后的有向图肯定没有圈，且最长路包含的点数不超过 p （想一想，为什么），所以直观上， p 应该是越小越好。

事实上，可以证明^①当 p 取最小值时，最长路恰好包含 p 个结点，而且这个结果是所有定向方案中最优的。这样，就成功地把问题转化为了“结点分层”问题，而这个“结点分层”问题实际上就是之前学过的色数问题：把图中的结点染成尽量少的颜色，使得相邻结点颜色不同。套用前面学过的动态规划算法，在 $O(3^k)$ 时间内即解决了问题，其中 $k \leq 15$ ，为资源的最大数目。

本题是关于“建模与问题转换”的一道经典问题，请读者仔细体会。

例题 9-29 整数传输 (Integer Transmission, ACM/ICPC Beijing 2007, UVa1228)

你要在一个仿真网络中传输一个 n 比特的非负整数 k 。各比特从左到右传输，第 i 个比特的发送时刻为 i 。每个比特的网络延迟总是为 $0 \sim d$ 之间的实数（因此从左到右第 i 个比特的到达时刻为 $i \sim i+d$ 之间）。若同时有多个比特到达，实际收到的顺序任意。求实际收到的整数有多少种，以及它们的最小值和最大值。例如， $n=3$ ， $d=1$ ， $k=2$ （二进制为 010）时实际收到的整数的二进制可能是 001(1)、010(2)和 100(4)。 $1 \leq n \leq 64$ ， $0 \leq d \leq n$ ， $0 \leq k < 2^n$ 。

【分析】

为了简化问题，首先可以规定：所有 0 按照原来的顺序依次收到，所有的 1 也按照原来的顺序依次收到，只是 0 和 1 可能交错。这个规定非常重要，请读者仔细体会。

最小值和最大值可以用贪心法得到（留给读者思考），关键在于统计可能收到的整数数目。给定一个整数 P ，如何判断它是否可能被收到呢？来看一个例子。

例如， $k=11001010$ ， $d=3$ ，需要判断 $P=00111001$ 是否可以得到。一共有 8 个比特，则发送时刻为 1~8，接收时刻是 1~12。不难发现，接收时刻可以限制为 1~8，因为同一时刻接收的比特可以任意排列，所以把一个比特延迟到时刻 9~12 不会有任何好处。可以手算出一种方案，如图 9-25 所示。

^① 证明思路是从定向方案构造分层图。先把所有路径的起点作为第 0 层。

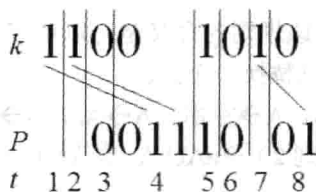


图 9-25 手算方案

上图的意思是： k 的比特 1 和比特 2 均延迟到时刻 4，比特 7 延迟到时刻 8。不难发现，对于任意给定的 P ，都可以用贪心法求解：从左到右依次考虑 P 的每一个比特。如果是 0，则接收 k 中没有收到的最左边的 0；如果是 1，则接收 k 中没有收到的最左边的 1。

仔细推敲这个过程，可以得到一个结论：在任意时刻， k 中已收到的比特中最右边的那个比特一定没有延迟（理论上可以延迟，但不会得到更优的解）。如图 9-26 所示， $k=111011001110$ ，框中的比特是已收到的比特，则最右边那个已收到比特（即左数第 3 个 0）无延迟，即接收时刻和发送时刻均为 8。

这样就可以动态规划了^①：用 $d(i, j)$ 表示 k 的前 i 个 0 和前 j 个 1 收到以后可能形成的整数个数，则只有两种转移方式：

如果下一个收到的比特可以是 0，则 $d(i+1, j)$ 需要加上 $d(i, j)$ 。

如果下一个收到的比特可以是 1，则 $d(i, j+1)$ 需要加上 $d(i, j)$ 。

所以问题的关键就是：如何判断下一个收到的比特是否可以为 0 或者 1？还是刚才那个例子，因为已经收到了 3 个 0 和 4 个 1，所以状态是 $d(3, 4)$ 。假设下一个收到的比特是 0，则左数第 5 个 1（发送时刻为 6）至少得延迟到第 4 个 0 的发送时刻（即时刻 12），如图 9-27 所示。如果 $d < 12 - 6 = 6$ ，说明假设不成立。

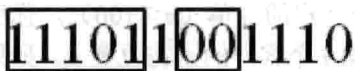


图 9-26 已收到的比特



图 9-27 判断收到的比特

一般地，设第 i 个 0 的发送时刻为 Z_i ，第 i 个 1 的发送时刻为 O_i ，则当且仅当 $O_j + d \leq Z_{i+1}$ 时 $d(i, j)$ 可以转移到 $d(i+1, j)$ ，即下一个收到的比特为 0。同理，当且仅当 $Z_i + d \geq O_{j+1}$ 时， $d(i, j)$ 可以转移到 $d(i, j+1)$ 。

状态有 $O(n^2)$ 个，每个状态只有两个决策，因此总时间复杂度为 $O(n^2)$ 。

例题 9-30 给孩子起名 (The Best Name for Your Baby, ACM/ICPC Yokohama 2006, UVa1375)

给一个包含 n 条规则的上下文无关文法和长度 l ($1 \leq n \leq 50$, $0 \leq l \leq 20$)，求出满足该文法的串中，长度恰好为 l 的字典序最小串。如果不存在，输出单个字符 “-”。

“满足文法”是指可以不断使用规则，把单个大写字母 S 变成这个串。每条规则形如 $A \rightarrow a$ ，其中 A 是一个大写字母（表示非终结符）， a 是一个由大小写字母组成的字符串（长

^① 准确地说这不是动态规划，而是组合数学中的递推，因为本题不是最优化问题，而是计数问题。不过解决两个问题的思路是相同的，所以很多人把组合数学中的递推也算作动态规划。

度不超过 10，且可以为空串）。该规则的含义是可以用字符串 a 来替换当前字符串中的大写字母 A（如果有多个 A，每次只替换一个）。

例如，有 4 条规则： $S \rightarrow aAB$ ， $A \rightarrow \text{空串}$ ， $A \rightarrow Aa$ ， $B \rightarrow AbbA$ ，那么 aabb 满足该文法，因为 $S \rightarrow aAB$ （规则 1） $\rightarrow aB$ （规则 2） $\rightarrow aAbbA$ （规则 4） $\rightarrow aAabbA$ （规则 3） $\rightarrow aAabb$ （规则 2） $\rightarrow aabb$ （规则 2）。

【分析】

题目中的文法比较复杂，首先把它们简化一下。例如 $S \rightarrow ABaA$ 拆成 3 个： $S \rightarrow AP_1$ ， $P_1 \rightarrow BP_2$ ， $P_2 \rightarrow aA$ 。这样，所有规则都变成了 $A \rightarrow BC$ 的形式，规则总数不超过 $50 \times 10 = 500$ 。为了叙述方便，文法拆分后的所有大小字母和小写字母统称为符号。

接下来试着动态规划： $d(i, L)$ 表示符号 i 能变成的、长度为 L 且字典序最小的串。如果符号 i 不能变成长度为 L 的串，则 $d(i, L)$ 无定义。例如，符号 i 是小写字母且 L 不等于 1 时， $d(i, L)$ 无定义。是不是可以这样转移呢：

$$d(i, L) = \min\{d(j, p) + d(k, L-p) \mid \text{存在规则 } i \rightarrow jk, 0 \leq p \leq L\}$$

逻辑上没问题，但是如果直接写一个记忆化搜索，程序可能会无限递归：如果有两个规则 $A \rightarrow BC$ ， $B \rightarrow AC$ ，则 $d(A, L)$ 的计算需要调用 $d(B, L)$ ，而计算 $d(B, L)$ 时又会调用 $d(A, L)$ ……

怎么办呢？注意到上述情况只有 $p=0$ 或者 $p=L$ 时才会出现，大多数情况下还是可以按照 L 从小到大的顺序计算的。所以可以特殊处理 L 相同时的所谓“同层状态转移”。

具体做法如下：首先从小到大枚举 L 。对于给定的 L ，先只考虑 $0 < p < L$ ，计算出所有 $d(i, L)$ 的中间结果，然后把这 $d(i, L)$ 从小到大排序。对于每个 $d(i, L)$ ，看看是否有状态 j 满足： $d(j, 0)$ 有定义（此时它肯定是空串），并且存在规则 $S \rightarrow ij$ 或者 $S \rightarrow ji$ 。如果存在，用 $d(i, L)$ 更新 $d(S, L)$ 。这个过程类似第 11 章中将要介绍的 Dijkstra，请读者仔细体会。

例题 9-31 送匹萨（Pizza Delivery, ACM/ICPC Daejeon 2012, UVa1628）

你是一个匹萨店的老板，有一天突然收到了 n 个客户的订单（ $n \leq 100$ ）。你所在的小镇只有一条笔直的大街，其中位置 0 是你的匹萨店，第 i 个客户的家在位置 p_i 。如果你选择给第 i 个客户送餐，他将会支付你 $e_i - t_i$ 元，其中 t_i 是你到达他家的时刻。当然，如果你到的太晚，使得 $e_i - t_i < 0$ ，你可以路过他家但是不去给他送餐，免得他反过来找你要钱。

你只有一个送餐车，因此只能往返地送餐，如图 9-28 所示就是一个路线。图中的第一行是位置，第二行是 e_i 。图上的路线对应的总收益为 12（ c_4 付 3 元， c_2 付 3 元， c_5 付 5 元， c_1 付 1 元）。

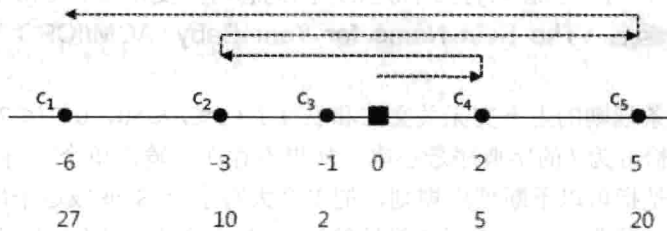


图 9-28 送餐路线

不过图 9-28 所示路线并不是最优的。最优路线是 $0 \rightarrow c_3 \rightarrow c_2 \rightarrow c_1 \rightarrow c_5$ ，总收益是 32。你

的任务是求出最大收益。

【分析】

本题是不是似曾相识？没错，本节开头的“修缮长城”一题和本题很像，但是有一个重大的不同：在本题中，可以“放弃”一些订单，所以无法像“修缮长城”那样规定“路过的点总是顺便修好”，也无法“准确地提前累加未来的费用”。如果要准确地判断每个客户是否有“未来费用”，必须记录当前时间，因为无法“提前知道”某个客户是否要送餐，只有等到达一个客户时发现收益变“负”，才会决定放弃它。

看上去很麻烦对吗？其实也不必过于沮丧。本题并不是纯粹的“加强版”，也有条件在本题中被弱化了。例如，所有客户的“单位时间罚款”是一样的，所以并不需要知道具体还有哪些客户没有到达，而只需要知道有多少客户没有到达。另一个弱化条件是： n 的范围变小，所以时间复杂度可以略有提高。如果本题的解法仍是动态规划，这就意味着每个状态的决策数可以增加，或者维数可以增加。

上述分析方式其实与动态规划本身并没有什么关系，但却是一种非常重要的思维过程，值得读者仔细体会。下面是本题的解法，建议读者自行思考片刻以后再看。

设 $d(i, j, k, p)$ 表示不考虑 $i \sim j$ 的客户（已经送过餐或者已经决定放弃），目前位置是 p ($p=0$ 表示在 i , $p=1$ 表示在 j)，还要给 k 个人送餐的最大收益。第一个送餐的人 i 以及送餐总人数 k 都需要枚举，最终答案是 $\max\{d(i, i, k-1, 0) + (e_i - |p_i|) * k \mid 1 \leq k \leq n\}$ ，这里的 $(e_i - |p_i|) * k$ 就是指从 0 到 p_i 的过程中，所有 k 个送餐客户的罚款总和。状态转移方程留给读者思考——对于已经阅读到这里的读者，相信这不难做到的。

9.6 训练参考

动态规划是算法竞赛的宠儿——几乎所有算法竞赛中都会出现动态规划的题目。本章虽然也包含一些知识点和理论讲解，但重中之重是那些经典题目（例如，LIS、LCS、最优矩阵链乘、树的重心和 TSP 等）和例题。本章的例题数量是本书目前为止最多的，难度也是最大的。建议读者先掌握不带星号的例题，然后逐步学习带一个星号的例题和两个星号的例题。有些例题比较复杂，甚至需要反复理解才能掌握。例题列表如表 9-2 所示。

表 9-2 例题列表

类 别	题 号	题目名称（英文）	备 注
例题 9-1	UVa1025	A Spy in the Metro	DAG 的动态规划
例题 9-2	UVa437	The Tower of Babylon	DAG 的动态规划
例题 9-3	UVa1347	Tour	经典问题
例题 9-4	UVa116	Unidirectional TSP	多段图的最短路；字典序最小解
例题 9-5	UVa12563	Jin Ge Jin Qu [h]ao	0-1 背包问题
例题 9-6	UVa11400	Lighting System Design	线性结构上的动态规划
例题 9-7	UVa11584	Partitioning by Palindromes	线性结构上的动态规划；优化
例题 9-8	UVa1625	Color Length	类似于 LCS 的动态规划；指标函数的分解

续表

类 别	题 号	题目名称（英文）	备 注
例题 9-9	UVa10003	Cutting Sticks	类似于最优矩阵链乘的动态规划
例题 9-10	UVa1626	Brackets Sequence	递归结构的动态规划
*例题 9-11	UVa1331	Minimax Triangulation	类似于最优三角剖分的动态规划
例题 9-12	UVa12186	Another Crisis	树形动态规划
例题 9-13	UVa1220	Party at Hali-Bula	树形动态规划；解的唯一性
例题 9-14	UVa1218	Perfect Service	树形动态规划；状态转移方程的优化
例题 9-15	UVa10817	Headmaster's Headache	集合的动态规划；位运算
例题 9-16	UVa1252	Twenty Questions	集合的动态规划；时间优化
*例题 9-17	UVa1412	Fund Management	复杂状态动态规划；和指标函数值有关的状态转移
例题 9-18	UVa10618	Tango Tango Insurrection	多阶段决策问题
例题 9-19	UVa1627	Team them up!	图论模型；0-1 背包
例题 9-20	UVa10934	Dropping water balloons	经典问题
例题 9-21	UVa1336	Fixing the Great Wall	动态规划中“未来费用”的计算
例题 9-22	UVa12105	Bigger is Better	用动态规划辅助其他算法
*例题 9-23	UVa1204	Fun Game	字符串集合的动态规划
*例题 9-24	UVa12099	Bookcase	类似 0-1 背包问题的动态规划；状态优化
*例题 9-25	UVa12170	Easy Climb	最优解的特征分析；用单调队列优化动态规划
*例题 9-26	UVa1380	A Scheduling Problem	树的动态规划（复杂）
**例题 9-27	UVa10559	Blocks	给状态增加维度
*例题 9-28	UVa1439	Exclusive Access 2	图论模型；Dilworth 定理
**例题 9-29	UVa1228	Integer Transmission	深入分析问题
**例题 9-30	UVa1375	The Best Name for Your Baby	上下文无关文法；有“环”的动态规划
**例题 9-31	UVa1628	Pizza Delivery	深入分析问题

下面是一些形形色色的动态规划问题，难度各异。建议读者阅读所有题目，然后认真思考每一道题。对于能写出状态转移方程的题目，尽量编程提交。

习题 9-1 最长的滑雪路径（Longest Run on a Snowboard, UVa 10285）

在一个 $R \times C$ ($R, C \leq 100$) 的整数矩阵上找一条高度严格递减的最长路。起点任意，但每次只能沿着上下左右 4 个方向之一走一格，并且不能走出矩阵外。如图 9-29 所示，最长路就是按照高度 25, 24, 23, ..., 2, 1 这样走，长度为 25。矩阵中的数均为 0~100。

	1	2	3	4	5
16	17	18	19	6	
15	24	25	20	7	
14	23	22	21	8	
13	12	11	10	9	

图 9-29 最长路径示例

习题 9-2 免费糖果（Free Candies, UVa 10118）

桌上有 4 堆糖果，每堆有 N ($N \leq 40$) 颗。佳佳有一个最多可以装 5 颗糖的小篮子。他

每次选择一堆糖果，把最顶上的一颗拿到篮子里。如果篮子里有两颗颜色相同的糖果，佳佳就把它们从篮子里拿出来放到自己的口袋里。如果篮子满了而里面又没有相同颜色的糖果，游戏结束，口袋里的糖果就归他了。当然，如果佳佳足够聪明，他有可能把堆里的所有糖果都拿走。为了拿到尽量多的糖果，佳佳该怎么做呢？

习题 9-3 切蛋糕 (Cake Slicing, ACM/ICPC Nanjing 2007, UVa1629)

有一个 n 行 m 列 ($1 \leq n, m \leq 20$) 的网格蛋糕上有一些樱桃。每次可以用一刀沿着网格线把蛋糕切成两块，并且只能够直切不能拐弯。要求最后每一块蛋糕上恰好有一个樱桃，且切割线总长度最小。如图 9-30 所示是一种切割方法。

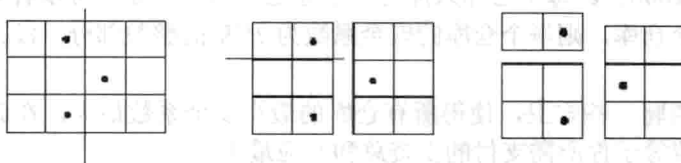


图 9-30 蛋糕切法示例

习题 9-4 串折叠 (Folding, ACM/ICPC NEERC 2001, UVa1630)

给出一个由大写字母组成的长度为 n ($1 \leq n \leq 100$) 的串，“折叠”成一个尽量短的串。例如，AAAAAAAAABABABCCD 折叠成 9(A)3(AB)CCD。折叠是可以嵌套的，例如，NEERCYESYESYESNEERCYESYESYES 可以折叠成 2(NEERC3(YES))。多解时可以输出任意解。

习题 9-5 邮票和信封 (Stamps and Envelope Size, ACM/ICPC World Finals 1995, UVa242)

假定一张信封最多贴 5 张邮票，如果只能贴 1 分和 3 分的邮票，可以组成面值 1~13 以及 15，但不能组成面值 14。我们说：对于邮票组合 {1,3} 以及数量上限 $S=5$ ，最大连续邮资为 13。1~13 和 15 的组成方法如表 9-3 所示。

表 9-3 1~3 和 15 的组成方法

1=1	2=1+1	3=3	4=1+3	5=1+1+3
6=3+3	7=1+3+3	8=1+1+3+3	9=3+3+3	10=1+3+3+3
11=1+1+3+3+3	12=3+3+3+3	13=1+3+3+3+3	14 无法表示	15=3+3+3+3+3

输入 S ($S \leq 10$) 和若干邮票组合 (邮票面值不超过 100)，选出最大连续邮资最大的一个组合。如果有多个并列，邮票组合中邮票的张数应最多。如果还有并列，邮票从大到小排序后字典序应最大。

习题 9-6 电子人的基因 (Cyborg Genes, UVa 10723)

输入两个 A~Z 组成的字符串 (长度均不超过 30)，找一个最短的串，使得输入的两个串均是它的子序列 (不一定连续出现)。你的程序还应统计长度最短的串的个数。例如，ABAAXGF 和 AABXFGA 的最优解之一为 AABAAXGFGA，一共有 9 个解。

习题 9-7 密码锁 (Locker, Tianjin 2012, UVa1631)

有一个 n ($n \leq 1000$) 位密码锁，每位都是 0~9，可以循环旋转。每次可以让 1~3 个相

邻数字同时往上或者往下转一格。例如, 567890→567901 (最后3位向上转)。输入初始状态和终止状态 (长度不超过1000), 问最少要转几次。例如, 111111 到 222222 至少转2次, 由 896521 到 183995 则要转12次。

习题 9-8 阿里巴巴 (Alibaba, ACM/ICPC SEERC 2004, UVa1632)

直线上有 n ($n \leq 10000$) 个点, 其中第 i 个点的坐标是 x_i , 且它会在 d_i 秒之后消失。Alibaba 可以从任意位置出发, 求访问完所有点的最短时间。无解输出 No solution。

习题 9-9 仓库守卫 (Storage Keepers, UVa10163)

你有 n ($n \leq 100$) 个相同的仓库。有 m ($m \leq 30$) 个人应聘守卫, 第 i 个应聘者的能力值为 P_i ($1 \leq P_i \leq 1000$)。每个仓库只能有一个守卫, 但一个守卫可以看守多个仓库。如果应聘者 i 看守 k 个仓库, 则每个仓库的安全系数为 P_i/k 的整数部分。没人看守的仓库安全系数为 0。

你的任务是招聘一些守卫, 使得所有仓库的最小安全系数最大, 在此前提下守卫的能力值总和 (这个值等于你所需支付的工资总和) 应最小。

习题 9-10 照亮体育馆 (Barisal Stadium, UVa10641)

输入一个凸 n ($3 \leq n \leq 30$) 边形体育馆和多边形外的 m ($1 \leq m \leq 1000$) 个点光源, 每个点光源都有一个费用值。选择一组点光源, 照亮整个多边形, 使得费用值总和尽量小。如图 9-31 所示, 多边形 ABCDEF 可以被两组光源 {1,2,3} 和 {4,5,6} 照亮。光源的费用决定了哪组解更优。

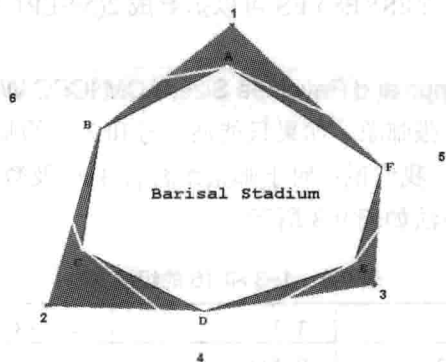


图 9-31 被点光源照亮的多边形

习题 9-11 禁止的回文子串 (Dyslexic Gollum, ACM/ICPC Amritapuri 2012, UVa1633)

输入正整数 n 和 k ($1 \leq n \leq 400$, $1 \leq k \leq 10$), 求长度为 n 的 01 串中有多少个不含长度至少为 k 的回文连续子串。例如, $n=k=3$ 时只有 4 个串满足条件: 001, 011, 100, 110。

习题 9-12 保卫 Zonk (Protecting Zonk, ACM/ICPC Dhaka 2006, UVa12093)

给定一个有 n ($n \leq 10000$) 个结点的无根树。有两种装置 A 和 B, 每种都有无限多个。

- 在某个结点 X 使用 A 装置需要 $C1$ ($C1 \leq 1000$) 的花费, 并且此时与结点 X 相连的边都被覆盖。
- 在某个结点 X 使用 B 装置需要 $C2$ ($C2 \leq 1000$) 的花费, 并且此时与结点 X 相连的边以及与结点 X 相连的点相连的边都被覆盖。

求覆盖所有边的最小花费。

习题 9-13 叠盘子 (Stacking Plates, ACM/ICPC World Finals 2012, UVa1289)

有 n ($1 \leq n \leq 50$) 堆盘子, 第 i 堆盘子有 h_i 个盘子 ($1 \leq h_i \leq 50$), 从上到下直径不减。所有盘子的直径均不超过 10000。有如下两种操作。

- split: 把一堆盘子从某个位置处分成上下两堆。
- join: 把一堆盘子 a 放到另一堆盘子 b 的顶端, 要求是 a 底部盘子的直径不超过 b 顶端盘子的直径。

你的任务是用最少的操作把所有盘子叠成一堆。

习题 9-14 圆和多边形 (Telescope, ACM/ICPC Tsukuba 2000, UVa1543)

给你一个圆和圆周上的 n ($3 \leq n \leq 40$) 个不同点。请选择其中的 m ($3 \leq m \leq n$) 个, 按照在圆周上的顺序连成一个 m 边形, 使得它的面积最大。例如, 在图 9-32 中, 右上方的多边形最大。

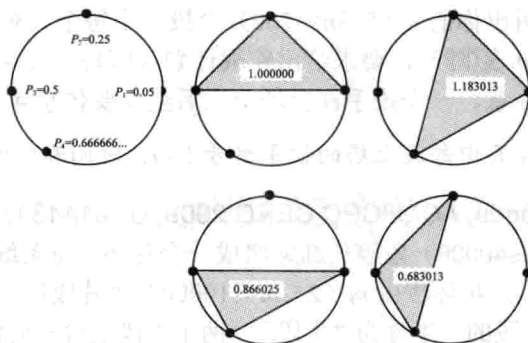


图 9-32 圆和多边形问题示意图

习题 9-15 学习向量 (Learning Vector, ACM/ICPC Dhaka 2012, UVa12589)

输入 n 个向量 (x, y) ($0 \leq x, y \leq 50$), 要求选出 k 个, 从 $(0,0)$ 开始画, 使得画出来的折线与 x 轴围成的图形面积最大。例如, 4 个向量是 $(3,5), (0,2), (2,2), (3,0)$, 可以依次画 $(2,2), (3,0), (3,5)$, 围成的面积是 21.5, 如图 9-33 所示。输出最大面积的两倍。 $1 \leq k \leq n \leq 50$ 。

习题 9-16 野餐 (The Picnic, ACM/ICPC NWERC 2002, UVa1634)

输入 m ($m \leq 100$) 个点, 选出其中若干个点, 以这些点为顶点组成一个面积最大的凸多边形, 使得内部没有输入点 (边界上可以有)。输入点的坐标各不相同, 且至少有 3 个点不共线, 如图 9-34 所示。

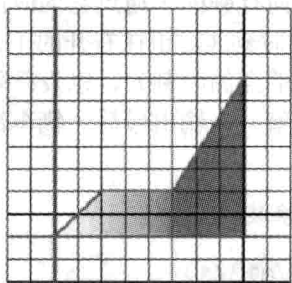


图 9-33 向量所围面积

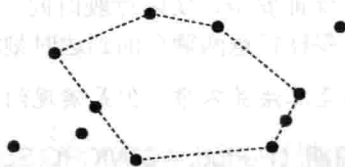


图 9-34 输入点

习题 9-17 佳佳的筷子 (Chopsticks, UVa 10271)

中国人吃饭喜欢用筷子。佳佳与常人不同，他的一套筷子有 3 只，两根短筷子和一只比较长的（一般用来穿香肠之类的食物）。两只较短的筷子的长度应该尽可能接近，但是最长那只的长度无须考虑。如果一套筷子的长度分别是 A, B, C ($A \leq B \leq C$)，则用 $(A-B)^2$ 的值表示这套筷子的质量，这个值越小，这套筷子的质量越高。

佳佳请朋友吃饭，并准备为每人准备一套这种特殊的筷子。佳佳有 N ($N \leq 1000$) 只筷子，他希望找到一种办法搭配好 $K+8$ 套筷子，使得这些筷子的质量值和最小。保证筷子足够，即 $3K+24 \leq N$ 。

提示：需要证明一个猜想。

习题 9-18 棒球投手 (Pitcher Rotation, ACM/ICPC Kaosiung 2006, UVa1379)

你经营着一支棒球队。在接下来的 $g+10$ 天中会有 g ($3 \leq g \leq 200$) 场比赛，其中每天最多一场比赛。你已经分析出你的 n ($5 \leq n \leq 100$) 个投手中每个人对阵所有 m ($3 \leq m \leq 30$) 个对手的胜率（一个 $n \times m$ 矩阵），要求给出作战计划（即每天使用哪个投手），使得总获胜场数的期望值最大。注意，一个投手在上场一次后至少要休息 4 天。

提示：如果直接记录前 4 天中每天上场的投手编号 $1 \sim n$ ，时间和空间都无法承受。

习题 9-19 花环 (Garlands, ACM/ICPC CERC 2009, UVa1443)

你的任务是用 n ($n \leq 40000$) 条等长细绳组成一个花环。每条细绳上都有一颗珍珠，重量为 w_i ($1 \leq w_i \leq 10000$)。花环应由 m ($2 \leq m \leq 10000$) 个片段组成，每个片段必须包含连续的偶数条细绳。每个片段的一半称为“半段”（两个半段包含相同数量的细绳），每个“半段”最多能有 d ($1 \leq d \leq 10000$) 条细绳。你的任务是让最重的半段尽量轻。如图 9-35 所示，12 条细绳的最优解是如下的 3 个片段，最重的半段的重量为 6（左数第 1, 4, 6 个半段）。

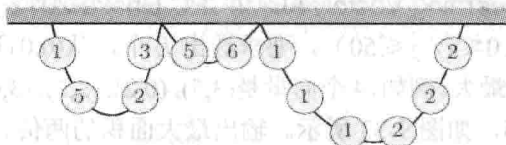


图 9-35 12 条细绳的最优解

习题 9-20 山路 (Mountain Road, NWERC 2009, UVa12222)

有一条狭窄的山路只有一个车道，因此不能有两辆相反方向的车同时驶入。另外，为了确保安全，对于山路上的任意一点，相邻的两辆同向行驶的车通过它的时间间隔不能少于 10 秒。给定 n ($1 \leq n \leq 200$) 辆车的行驶方向、到达时刻（对于往右开的车来说是到达山路左端点的时刻，而对于往左开的车来说是指到达右端点的时刻），以及行驶完山路的最短时间（为了保证安全，实际行驶时间可以高于这个值），输出最后一辆车离开山路的最早时刻。输入保证任意两辆车的到达时刻均不相同。

提示：本题的主算法并不难，但是实现细节需要仔细推敲。

习题 9-21 周期 (Period, ACM/ICPC Seoul 2006, UVa1371)

两个串的编辑距离为进行的修改、删除和插入操作次数的最小值（每次一个字符）。

如图 9-36 所示, $A=abcdefg$ 和 $B=ahcefig$ 的编辑距离为 3。

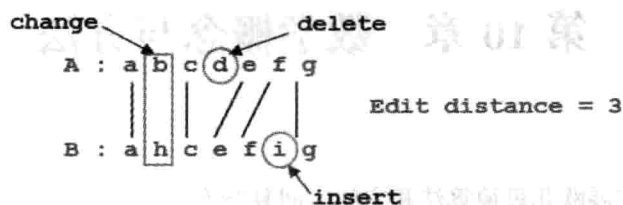


图 9-36 编辑距离

如果 x 可以分成若干部分, 使得每部分和 y 的编辑距离都不超过 k , 则 y 是 x 的 k -近似周期。例如, $x=abcdabcabb$, $y=abc$, x 可以分解为 $abcd+abc+abb$, 3 部分和 y 的编辑距离分别为 1, 0, 1, 因此 y 是 x 的 1-近似周期。

输入由小写字母组成的 x 和 y , 求最小的 k 使得 y 是 x 的 k -近似周期。 $|y| \leq 50$, $|x| \leq 5000$ 。

提示: 直接想出的动态规划算法很可能太慢, 要想办法降低时间复杂度。

习题 9-22 俄罗斯套娃 (Matryoshka, ACM/ICPC World Finals 2013, UVa 1579)

桌上有 n ($n \leq 500$) 个套娃排成一行, 你的任务是把它们套成若干个套娃组, 使得每个套娃组内的套娃编号恰好是从 1 开始的连续编号。操作规则如下:

- 只能把小的套在大的里面, 大小相等的套娃相互不能套。
- 每次只能把两个相邻的套娃组合成一个套娃组。
- 一旦有两个套娃属于同一个组, 它们永远都属于同一个组 (只有与相邻组合并的过程中会临时拆散)。

执行合并操作的前后, 所有套娃都是关闭的。为了合并两个套娃组, 你需要交替地把一些套娃打开、重新套起来、关闭。例如, 为了合并 $[1, 2, 6]$ 和 $[4]$, 需要打开套娃 6 和 4; 为了合并 $[1, 2, 5]$ 和 $[3, 4]$, 需要打开套娃 5, 4, 3 (只有先打开 4 才能打开 3)。要求打开/关闭的总次数最少。无解输出 impossible。例如, “1 2 3 2 4 1 3” 需要打开 7 次, 如表 9-4 所示。

表 9-4 “1 2 3 2 4 1 3” 需打开 7 次

操作前	操作后	打开的套娃
1 2 3 2 4 1 3	[1 2] 3 2 4 1 3	2
[1 2] 3 2 4 1 3	[1 2 3] 2 4 1 3	3
[1 2 3] 2 4 1 3	[1 2 3] [2 4] 1 3	4
[1 2 3] [2 4] 1 3	[1 2 3] [2 4 1] 3	4, 2
[1 2 3] [2 4 1] 3	[1 2 3] [2 4 1 3]	4, 3

习题 9-23 优化最大值电路 (Minimizing Maximizer, ACM/ICPC CERC 2003, UVa 1322)

所谓 Maximizer, 就是一个 n 输入 1 输出的硬件电路, 它可以用若干个串行 Sorter 来实现, 其中每个 Sorter(i, j) 表示把第 $i \sim j$ 个输入从小到大排序。最后一个 Sorter 的第 n 个输出就是整个 Maximizer 的输出。输入一个由 m 个 Sorter 组成的 Maximizer, 保留尽量少的 Sorter (顺序不变), 使得 Maximizer 仍能正常工作。 $n \leq 50000$, $m \leq 500000$ 。